TUM School of Computation, Information and Technology
Technical University of Munich



In collaboration with Agile Robots SE

Master's Thesis in Robotics, Cognition, Intelligence

Learning Long-Horizon Cyclic Nut Threading with a Dexterous Hand via Hybrid-Teacher Policy Distillation

Lernen langfristiger zyklischer Mutter-Gewindemontage mit einer geschickten Hand durch Policy-Destillation eines hybriden Lehrers

Supervisor	Prof. Dr.-Ing. habil. Alois C. Knoll
Advisor	Chi Zhang, M.Sc.
Industry supervisor	Antonio Gonzales Marin, M.Sc. Agile Robots SE
Author	Carsten Oertel
Date	November 15, 2026 in Munich

Disclaimer

I confirm that this Master's Thesis is my own work and I have documented all sources and material used.

Munich, November 15, 2026

(Carsten Oertel)

Acknowledgments

This thesis was conducted in collaboration with Agile Robots SE, Munich. The author was employed at Agile Robots SE during the course of this work, and the research was carried out using computational and hardware resources provided by Agile Robots SE, including the Franka Research 3 robot arm, the Inspire RH56DFX dexterous hand, and GPU infrastructure for reinforcement learning training. The author gratefully acknowledges Agile Robots SE for their support, in particular Antonio Gonzales Marin, M.Sc., for his supervision on the industry side.

The author further thanks Prof. Dr.-Ing. habil. Alois C. Knoll for supervising this thesis at the Chair of Robotics, Artificial Intelligence and Real-time Systems of the Technical University of Munich, and Chi Zhang, M.Sc., for his advising and for his work on the hybrid teacher, the distillation stack, and the hardware experiments this thesis builds on.

Abstract

Threaded joints are central to assembly because they can be released again and because their threads are standardized. This thesis asks how a robot with a dexterous hand can learn to thread a nut. Threading poses three difficulties at once. The skill has to be learned in simulation and transferred to hardware, although thread contact is hard to simulate. The hand has a high-dimensional action space and many ways to solve the task. Finally, threading is a long-horizon cyclic task: the range of the wrist and the fingers limits each turn, so the hand has to grasp, turn, release, and re-grasp the nut cycle after cycle while errors compound. The thesis decomposes the task during data generation and unifies it at deployment. A privileged reinforcement-learning teacher learns the pinch-grasp turn in a simplified thread simulation, and scripted release and return motions chain its turns into multi-cycle demonstrations. A single visuomotor student is distilled from these demonstrations by conditional flow matching and acts on a metric point cloud from a fixed RGB-D camera and on proprioception. It is trained on simulated data only. A calibrated camera model, visual randomization, and a Cartesian action path shared by teacher, student, and robot are designed to carry it zero-shot to a physical dexterous hand-arm system. In simulation, the student completes all ten threading cycles in each of ten trials, with a mean cyclic tracking error of $-1.77°$, a lag of less than two degrees per cycle. On hardware, its zero-shot results on the training nut are TBD, and so is its transfer to nuts of other sizes and profiles.

Kurzfassung

Schraubverbindungen sind zentral für die Montage, weil sie wieder lösbar und ihre Gewinde genormt sind. Diese Arbeit untersucht, wie ein Roboter mit einer geschickten Hand das Aufschrauben einer Mutter lernen kann. Das Aufschrauben stellt drei Schwierigkeiten zugleich. Die Fähigkeit muss in der Simulation gelernt und auf die Hardware übertragen werden, obwohl sich der Gewindekontakt nur schwer simulieren lässt. Die Hand hat einen hochdimensionalen Aktionsraum und viele Möglichkeiten, die Aufgabe zu lösen. Schließlich ist das Aufschrauben eine zyklische Aufgabe mit langem Horizont: Der Bewegungsbereich von Handgelenk und Fingern begrenzt jede Drehung, sodass die Hand die Mutter Zyklus für Zyklus greifen, drehen, loslassen und erneut greifen muss, während sich Fehler aufsummieren. Die Arbeit zerlegt die Aufgabe bei der Datenerzeugung und vereint sie beim Einsatz. Ein privilegierter Lehrer lernt mit Reinforcement Learning die Drehung im Spitzgriff in einer vereinfachten Gewindesimulation, und skriptgesteuerte Loslass- und Rückkehrbewegungen verketten seine Drehungen zu Demonstrationen über mehrere Zyklen. Aus diesen Demonstrationen wird mittels bedingtem Flow Matching eine einzige visuomotorische Schülerpolicy destilliert, die auf einer metrischen Punktwolke einer festen RGB-D-Kamera und auf Propriozeption arbeitet. Sie wird ausschließlich mit simulierten Daten trainiert. Ein kalibriertes

Kameramodell, visuelle Randomisierung und ein von Lehrer, Schüler und Roboter geteilter kartesischer Aktionspfad sollen sie ohne Nachtraining (zero-shot) auf ein reales geschicktes Hand-Arm-System übertragen. In der Simulation vollendet die Schülerpolicy in jedem von zehn Versuchen alle zehn Schraubzyklen mit einem mittleren zyklischen Nachführfehler von $-1{,}77°$, also einem Rückstand von weniger als zwei Grad pro Zyklus. Auf der Hardware sind ihre Zero-Shot-Ergebnisse auf der Trainingsmutter TBD, ebenso die Übertragung auf Muttern anderer Größen und Formen.

Contents

Acknowledgments **v**

List of Abbreviations **xvii**

List of Symbols **xix**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Problem: How Can a Robot Learn Threading? . . . 2
1.2.1 Simulation . . . 2
1.2.2 Threading with a Dexterous Hand . . . 3
1.2.3 Threading as a Long-Horizon Cyclic Task . . . 4
1.3 Approach . . . 5
1.4 Research Questions . . . 6
1.5 Contributions . . . 6
1.6 Outline . . . 7

2 State of the Art **9**
2.1 Theoretical Background . . . 9
2.1.1 Threaded Joints and Thread Geometry . . . 9
2.1.2 Reinforcement Learning . . . 9
2.1.3 Imitation Learning . . . 12
2.1.4 Action Chunking . . . 12
2.1.5 Generative Action Heads: Diffusion and Flow Matching . . . 13
2.1.6 Operational-Space Control . . . 14
2.2 Related Work . . . 17
2.2.1 Robotic Assembly and Threading . . . 17
2.2.2 Dexterous Manipulation with Learned Policies . . . 18
2.2.3 Visual Representations for Manipulation . . . 19
2.2.4 Long-Horizon Tasks and Policy Chaining . . . 19
2.2.5 Teacher–Student and Visuomotor Distillation . . . 20
2.2.6 Summary and Research Gap . . . 21

3 Methodology **23**
3.1 Problem Formulation . . . 24
3.2 Simulated and Physical System . . . 24
3.2.1 Platform . . . 25
3.2.2 Simplified Thread Simulation . . . 25
3.2.3 Action Contract and Controller . . . 26
3.3 Privileged Teacher Training . . . 27
3.4 Hybrid Teacher and Dataset Collection . . . 29

3.4.1 Cyclic Teacher Rollout in Simulation . . . 30
3.4.2 Disturbance Augmentation . . . 31
3.4.3 Vision Data Collection . . . 31
3.4.4 Dataset Format . . . 32
3.5 Student Policy Distillation by Conditional Flow Matching . . . 33
3.5.1 Conditional Flow Matching . . . 33
3.5.2 Student Architecture . . . 34
3.5.3 Offline Training . . . 36

4 Experiments in Simulation **37**
4.1 Experimental Setup . . . 37
4.2 Evaluation Protocol . . . 37
4.3 Student Ablations in Simulation . . . 38
4.4 Summary . . . 39

5 Experiments on Real Hardware **41**
5.1 Hardware Setup . . . 41
5.2 Zero-Shot Transfer Protocol . . . 42
5.3 Results on the Training Nut . . . 43
5.4 Generalization across Nut Sizes and Geometries . . . 43
5.5 Long-Horizon Stress . . . 44

6 Discussion **45**
6.1 Interpretation per Research Question . . . 45
6.2 Action and Observation Contracts . . . 46
6.3 What This Means for Robotic Assembly . . . 47
6.4 Limitations . . . 47

7 Conclusion and Future Work **49**
7.1 Conclusion . . . 49
7.2 Future Work . . . 50

A Implementation Details **51**
A.1 Simulation and Controller . . . 51
A.2 Teacher Reward . . . 51
A.3 Hybrid Teacher . . . 52
A.4 Teacher Hyperparameters . . . 52
A.5 Student Hyperparameters . . . 52
A.6 Dataset Summary . . . 53

Bibliography **55**

List of Figures

1.1 How threading is automated today: (a) a stationary nutrunner (image from [3]), (b) a multi-spindle station that tightens several fasteners of one part at once, (c) a robot with a screwdriving end effector, and (d) a fixture that presents the part in a fixed pose. Each solution is built for one fastener type and a known part pose. 2

1.2 How a physics engine resolves contact. Two bodies that overlap by a penetration depth d receive a repulsive force along the contact normal that grows with d and pushes them apart. 3

1.3 How interpenetration can be exploited in thread simulation. A reward for axial progress of the nut is collected on the helical path by turning the nut, but also on a shortcut in which the nut interpenetrates the thread flank and crosses the crest into the next groove. 3

1.4 Overview of the approach: decompose during data generation, unify at deployment. Top: a privileged RL teacher is trained in parallel simulated workcells. During data collection its rollouts are chained with scripted release-and-reset controllers into long-horizon demonstrations $\mathcal{D}_{\text{sim}}$. A conditional flow-matching student is distilled from them and acts from RGB and depth. Bottom: the phases of one cycle on the physical robot, approach, grasp, rotation, goal reaching, and release-and-reset, after which the hand returns for the next cycle. 5

2.1 Threads and thread geometry according to the ISO standards. Placeholder. . . 10

2.2 Diffusion and conditional flow matching for a bimodal action distribution in a two-dimensional toy action space (u_1, u_2). Generation time runs from left to right, from Gaussian noise p_0 to the demonstrated actions p_1, and each slice shows the action space at one point in time. (a) A diffusion model adds noise to demonstrated actions (gray path) and learns to reverse this process over many denoising steps along curved trajectories. The point clouds on the slices are intermediate denoising samples, colored by the mode they reach. (b) Conditional flow matching regresses a velocity field onto the straight path between a Gaussian sample and a data sample (dashed). Integrating the learned field (gray arrows on the slices at $\lambda = 0.25$, 0.5, and 0.75) from noise reaches either mode in a few ODE steps. 15

2.3 Impedance control, illustrated on a planar arm with three joints. (a) A virtual spring and damper with stiffness k_1, k_2, k_3, the diagonal entries of K_q, act directly on each joint, pulling it toward its reference q_{ref}. (b) A virtual spring and damper with stiffness k_x, k_y, k_φ and damping d_x, d_y, d_φ, the diagonal entries of K_p and K_d, act on the controlled end-effector frame along each task direction instead. The resulting wrench (F_x, F_y, M_z) is mapped to the joint torques τ_1, τ_2, τ_3 through J^{T}. 17

3.1 The pipeline in detail. Left: the privileged RL policy executes grasp, rotation, and goal reaching and a scripted controller executes release and reset. Chained in a loop, they form the hybrid teacher that generates the simulated demonstrations $\mathcal{D}_{\text{sim}}$. Right: the student encodes the aligned RGB-D observation into a 64-D point-cloud feature, fuses it with proprioception, the normalized global progress, and the process-phase encoding into a 256-D condition, and a conditional flow-matching Transformer predicts an eight-step action chunk. 23

3.2 The robot platform, rendered from the simulation models used in this work. In the assembled system the hand is mounted on the arm flange. 25

3.3 Simplified thread simulation. A passive revolute joint θ_t about the bolt axis is kinematically coupled to a position-controlled prismatic joint d_t through $d_t = k\theta_t$ with $k = -\rho/(2\pi)$, so that the nut advances by one pitch per revolution without explicit thread contacts. The finger–nut contacts remain fully simulated. 26

3.4 Data-collection loop of the hybrid teacher over K threading cycles. The learned policy (blue) owns approach, grasp, turn, and hold. A physical release gate on the nut rotation hands over to the scripted transitions (orange): four waypoints, which open the pinch and retreat, and the gated return. A completed return rebases the turning target and starts the next cycle, and a return that times out rejects the episode. Table A.4 lists the gate thresholds. Every policy, waypoint, and return segment receives one random arm impulse, and every executed command is recorded in the common 9-D label space. 31

3.5 Three draws of the texture randomization in the simulated workcell. The dome light and the textures of the floor and the tabletop change between draws, while the robot keeps its nominal appearance. 32

3.6 Simulated depth with and without augmentation, next to real depth. Left: the noise-free depth of one frame of a simulated demonstration, rendered by the 320×180 head camera. Middle: the same depth image after the full augmentation used in student training, with depth noise, quantization, and dropout of edge pixels, correlated holes, and isolated pixels. The augmentation raises the fraction of invalid pixels in this frame from 21.4 % to 47.3 %. Right: a depth frame streamed from the head camera of the physical robot, shown for comparison. All three images use the same color map, from about 0.8 m (dark blue) to 2 m (dark red), and black marks invalid pixels. 33

3.7 Student architecture. The aligned RGB-D observation is back-projected into a point cloud and encoded into a 64-D visual feature. The 29-D proprioception is encoded by an MLP. Late fusion concatenates the two with the visual feature first and projects to the 256-D condition, to which learned embeddings of the global trajectory progress and of the process phase are added. A non-causal trajectory Transformer predicts the flow velocity of all eight action tokens given the condition token and a sinusoidal embedding of the flow time. An auxiliary head regresses the nut position from the visual feature during training. 34

4.1 Hardware setup. Left: the FR3 arm with the Inspire RH56DFX hand, the bolts on their holders in front of the robot, and the fixed RealSense D415 on its stand. Right: the RGB image and the metric depth image of the D415 as the student receives them, after center-cropping and downscaling to 320×180. . . 37

5.1 One threading cycle on the physical robot. Top, from left to right: approach, grasp, rotation, goal reaching, and release with return to the reset pose. Bottom: top-down views of the angular scale under the bolt during grasp, rotation, and goal reaching. The scale is read at the end of every goal-reaching phase to compute the cyclic and ten-cycle tracking errors. 43

List of Tables

3.1 Actor and critic observations of the single-size M24 threading teacher. Noise is applied to the actor only. Joint torques are divided by the effort limits of 87 N m (joints 1–4) and 12 N m (joints 5–7). 28
3.2 Domain randomization during teacher training. $U(a, b)$ denotes uniform sampling. Multiplicative factors are sampled as $1 + U(0, \ell)$ or its reciprocal with equal probability. Joint torques are in units of the joint effort limits. 29

4.1 Simulation ablations on the M24 hexagonal nut over ten trials. Each defined entry reports mean ± standard deviation. Completed cycles are averaged across trials, cyclic-error statistics use completed cycles only, and the ten-cycle error compares the accumulated rotation with the 600° target. It is reported as "–" when no trial completes all ten cycles. Errors are signed in degrees: negative denotes lag and positive overshoot. 39

5.1 Real-world zero-shot ablations on the M24 hexagonal nut over ten trials, with the same conventions as Table 4.1. Ablations (c) and (d) are evaluated in simulation only. 43
5.2 Real-world zero-shot transfer across nut sizes and profiles over ten trials per nut. Values are mean ± standard deviation. The octagonal, rectangular, and round profiles and the M30 and M36 sizes are absent from the demonstrations. 44

A.1 Simulation settings, thread model, and simulated hand of Section 3.2. 51
A.2 Action decoding and controller of the shared Cartesian contract. The values are nominal. The per-episode randomization of the simulated controller is listed in Table 3.2. In the physical column, "same" marks values shared with simulation. 52
A.3 Reward terms of the threading teacher (3.8). Progress terms pay only for newly achieved progress. The weight of P^{wrench} rises from zero at the first initial-yaw curriculum level to one at its full range. 52
A.4 Thresholds and timings of the scripted transitions of the hybrid teacher (Section 3.4.1). 53
A.5 PPO hyperparameters of the privileged teacher (rl_games, asymmetric actor–critic). 53
A.6 Hyperparameters of the conditional flow-matching student. 54
A.7 Demonstration data of the M24 ten-cycle collection used for the reported students. 54

List of Abbreviations

ACT	Action Chunking with Transformers
ADR	Automatic Domain Randomization
BC	Behavior Cloning
CFM	Conditional Flow Matching
DAgger	Dataset Aggregation
DoF	Degree of Freedom
DP3	3D Diffusion Policy
DR	Domain Randomization
EMA	Exponential Moving Average
FR3	Franka Research 3
GAE	Generalized Advantage Estimation
IL	Imitation Learning
LSTM	Long Short-Term Memory
MDP	Markov Decision Process
MLP	Multilayer Perceptron
ODE	Ordinary Differential Equation
OSC	Operational-Space Control
PD	Proportional-Derivative
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimization
RGB-D	Color and Depth Image
RL	Reinforcement Learning
RMSE	Root Mean Square Error
SDF	Signed Distance Field
TCP	Tool Center Point

List of Symbols

Unless a section states otherwise, the symbols below keep the following meanings in all chapters.

Symbol	Meaning
Task	
$\mathcal{S}$	ordered phase sequence of one threading cycle
$m_t \in \mathcal{M} = \mathcal{M}_{\text{RL}} \cup \mathcal{M}_{\text{script}}$	active phase at step t; learned phases $\mathcal{M}_{\text{RL}}$, scripted phases $\mathcal{M}_{\text{script}}$
K	number of cycles per trial ($K = 10$)
$\Delta\varphi^*$	desired clockwise nut rotation per cycle (60°)
φ_k	unwrapped nut angle after the goal-reaching phase of cycle k
e_k, e_{ep}	cyclic and episodic (accumulated) tracking error
$C(\tau)$, $E(\tau)$	completed cycles and tracking cost of a trajectory τ
ρ	thread pitch
θ_t, d_t	revolute and prismatic coordinate of the simplified thread joint
Teacher	
$\boldsymbol{x}_t$	full simulator state
$\boldsymbol{z}_t^{\pi} = g_{\pi}(\boldsymbol{x}_t, \boldsymbol{\xi}_t)$	actor observation with observation noise $\boldsymbol{\xi}_t$
$\boldsymbol{h}_t$	recurrent hidden state of the actor
$\pi_{\text{T}}(\boldsymbol{a} \mid \boldsymbol{z}^{\pi}, \boldsymbol{h})$, $\bar{\pi}_{\text{T}}$	privileged RL teacher and its deterministic mean
κ_m	scripted controller of transition phase $m \in \mathcal{M}_{\text{script}}$
π_{HT}	hybrid teacher
$\boldsymbol{a}_t \in [-1, 1]^9$	normalized Cartesian-controller action (teacher); rescaled filtered target (student label)
r_t, $R_{i,t}$, $P_{j,t}$	shaped reward, reward terms, penalty terms
Student	
$\boldsymbol{y}_t = (\boldsymbol{O}_t^{\text{RGBD}}, \boldsymbol{s}_t, p_t, \boldsymbol{e}_t^m)$	student observation: RGB-D image, proprioception, progress, phase encoding
$\mathcal{P}_t$, $\boldsymbol{v}_t \in \mathbb{R}^{64}$	point cloud and visual feature
$\boldsymbol{c}_t \in \mathbb{R}^{256}$	fused condition vector
H	action horizon (chunk length, $H = 8$)
H_{e}	execution horizon (actions executed per policy query)
$\boldsymbol{U}_t^D = [\boldsymbol{u}_t, \ldots, \boldsymbol{u}_{t+H-1}]$	demonstrated action chunk
$\boldsymbol{U}_t^0 \sim \mathcal{N}(\boldsymbol{0}, I)$	Gaussian source sample
$\lambda \in [0, 1]$	flow time
$\boldsymbol{U}_t^{\lambda}$	interpolant between source and target chunk
$\boldsymbol{v}_{\theta}(\boldsymbol{U}^{\lambda}, \lambda \mid \boldsymbol{c})$	learned conditional velocity field
N	number of ODE integration steps
β	probability of teacher control during online DAgger
$\mathcal{D}_{\text{sim}}$, $\mathcal{D}$	simulated demonstration set and the training set formed from it
Control	
$\boldsymbol{q}$, $\dot{\boldsymbol{q}}$	joint positions and velocities
$\boldsymbol{\tau}$	joint torques
J, M, Λ	geometric Jacobian, joint-space inertia, task-space inertia
K_p, K_d	proportional and derivative gains

Chapter 1

Introduction

1.1 Motivation

Threaded fasteners such as a nut on a bolt join parts so that the joint can be released again, which makes them the standard choice wherever products are assembled in the field, maintained, adjusted, or taken apart [1], [2]. They appear in almost every industry. The halves of a cast engine housing are bolted together, airplane seats are mounted with threaded bolts, and the injector of a rocket engine is bolted onto the combustion chamber. Many valve and pipe connections consist of two threaded halves that are screwed together with a sealing material, so that the liquid stays inside, from the plumbing of a house to the water cooling loop of a gaming PC. These applications have the same needs. The product has to be manufacturable in modules, assembled in a defined process, and opened again without damage for inspection, maintenance, repair, and development.

The vast majority of people have also met threaded fasteners in everyday life, for example when assembling IKEA furniture or fixing household objects and appliances. For a human, the process is intuitive. You pick up a screw, find the hole, set the tip of a Philips screwdriver into the screw head, and turn. If the screw does not bite, you back off, straighten it in the hole, and turn again. Industry solves the same task differently. In industrial production, turning the fastener along the thread is done by dedicated equipment, as Figure 1.1 illustrates. Handheld and stationary nutrunners drive the fastener with a powered spindle, multi-spindle stations tighten several fasteners of one part at once, and robots carry screwdriving end effectors that feed and drive fasteners. None of these machines has to find the hole. A fixture clamps the part in a known pose, so the hole is always at the same place. The screw or nut arrives from a feeder in a known orientation, and the tool is matched to that one fastener type. This setup pays off in high-volume production. It stops working where parts vary, where the cell is not built around one product, and where batches are small. It also stops working in maintenance and disassembly, where the hole is wherever the part happens to be, as it is for a person kneeling next to a half-built wardrobe.

People handle the cases that dedicated equipment cannot. They pick up a nut, place it on a bolt, and turn it with their fingers, and they use the same hands to operate a wrench or a screwdriver. A robot with a dexterous, multi-fingered hand could in principle do the same. It could work with the parts and tools that were designed for people, in workcells that were designed for people, without a dedicated tool for every fastener [4], [5]. Learning-based control has made in-hand manipulation with such hands practical in simulation and on hardware [5], [6], [7]. Robots have fastened screws with manual screwdrivers [8], [9], and a single learned policy has manipulated a range of everyday tools with a dexterous hand [10]. The vision behind this thesis is a dexterous robot that helps in manufacturing and assembly in a world built for humans. Threading a nut by hand is one of the basic skills such a robot



(a)

(b) Placeholder: multi-spindle station

(c) Placeholder: robot with screwdriving end effector

(d) Placeholder: fixture presenting the part

Figure 1.1: How threading is automated today: (a) a stationary nutrunner (image from [3]), (b) a multi-spindle station that tightens several fasteners of one part at once, (c) a robot with a screwdriving end effector, and (d) a fixture that presents the part in a fixed pose. Each solution is built for one fastener type and a known part pose.

needs. For humans the skill is trivial, but it is not for robots. This is an instance of Moravec's paradox: it is comparatively easy to make computers perform well on tasks that people find intellectually demanding, and difficult to give them the perception and mobility skills of a small child [11]. Industrial fastening shows both sides of this paradox. A nutrunner on a fixture drives a fastener without difficulty, because the cell design has already solved the hard part: the part is held in a known pose and the fastener is presented in a known orientation. Without this structure, a robot has to find the hole, align the fastener, and engage the thread, which a person does without thinking. If a dexterous robot is to thread, it has to learn the skill, and the next section explains why that is hard.

1.2 Problem: How Can a Robot Learn Threading?

Teaching a robot to thread a nut with a dexterous hand raises three difficulties at once. First, the skill has to be learned in simulation and transferred to hardware, although thread contact is very hard to simulate. Second, the dexterous hand has a high-dimensional action space and countless ways to solve the task. Third, threading is a long-horizon cyclic task. The following subsections treat the three difficulties in turn, and each ends with the requirement it puts on the method.

1.2.1 Simulation

Learning a contact-rich skill directly on hardware is slow and unsafe. Reinforcement learning (RL) needs many trials, every trial costs robot time and wear, and exploration produces collisions and excessive forces [12]. Simulation removes these limits. GPU-parallel simulators run thousands of environments at once and have made RL feasible for dexterous manipulation and for contact-rich assembly [7], [13], [14].

Thread contact, however, is among the hardest interactions to simulate. Engaged threads produce thousands of simultaneous contacts, and simulating them faithfully needs dedicated contact methods, such as the signed-distance-field contacts with which Factory simulates 1000 nut-and-bolt assemblies in real time on one GPU [14].

The GPU-parallel engines that make this scale possible, such as PhysX and MuJoCo, model contact through interpenetration. When two bodies overlap, the solver measures the penetration depth and generates a repulsive force that pushes the bodies apart again (Figure 1.2). Contact is therefore never strictly rigid. It tolerates a small overlap, and the repulsive force grows with that overlap.

For large bodies this tolerance is harmless. For threads it is not, because the thread

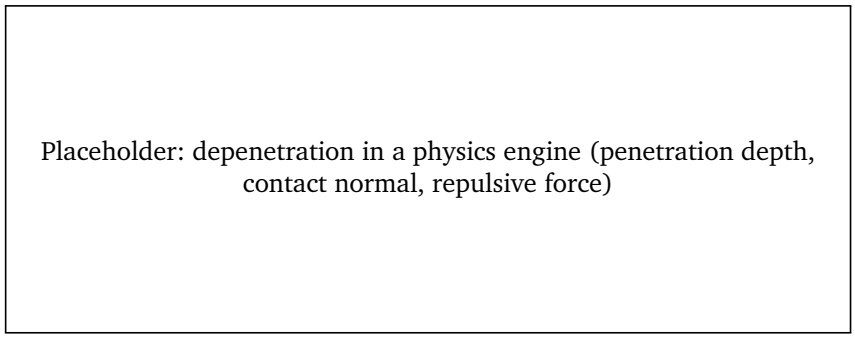


Figure 1.2: How a physics engine resolves contact. Two bodies that overlap by a penetration depth d receive a repulsive force along the contact normal that grows with d and pushes them apart.

geometry is small. The pitch is 3 to 4 mm for the nuts used in this thesis, and the flanks and crests are smaller still. At this scale, the overlap that the solver tolerates is no longer negligible against the geometry. A nut that is pushed along the axis can sink into the opposing flank and slide across the crest into the next groove, instead of following the helix. The repulsive force does not forbid this path, it only makes it more expensive. In simulation, the nut can thus phase through the threads.

A learning policy finds such shortcuts. If the reward pays for axial progress of the nut, the policy can collect it by pushing the nut through the threads rather than by turning it along the helix (Figure 1.3). The resulting behavior solves the simulated task and fails on hardware, where the threads are rigid and the nut advances only by rotating. Faithful thread contact therefore needs dedicated contact methods and is easy to exploit, and any simplification of the thread opens a gap in contact between simulation and reality. A deployed policy also has to perceive the nut through a camera, and rendered images differ from real ones, which opens a second gap in perception. Collecting real data to close these gaps would bring back the cost that simulation was meant to avoid.

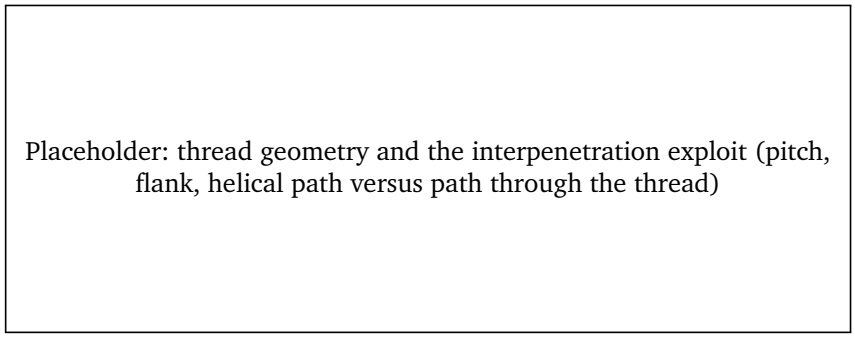


Figure 1.3: How interpenetration can be exploited in thread simulation. A reward for axial progress of the nut is collected on the helical path by turning the nut, but also on a shortcut in which the nut interpenetrates the thread flank and crosses the crest into the next groove.

Requirement: a thread model that is cheap enough for massively parallel RL and cannot be exploited through interpenetration, and a transfer recipe that carries the learned skill to hardware without real-world data.

1.2.2 Threading with a Dexterous Hand

A multi-fingered hand on an arm has far more actuated joints than a parallel-jaw gripper, and its action space is correspondingly high-dimensional [6], [15]. The task also has a large

solution space. People thread a nut with a single finger, with a pinch between thumb and index finger, or with a grasp of several fingers, depending on how strongly the thread resists. A policy that may choose among all of these strategies has to explore a vast space of finger motions, and a reward that steers it toward one reliable strategy is hard to design [12].

A dexterous hand also has to acquire the grasp, keep it under load while the nut turns, and release it again. Learned assembly with parallel-jaw grippers does not face this part of the problem. There, the threading or insertion policy starts every episode with the part already held, so it learns to move a part that is grasped, and grasping, holding, and re-grasping are not part of what it learns [14], [16], [17]. Learned dexterous threading has so far addressed the spinning of an already engaged fastener [18] and goal-conditioned screwing of furniture parts [19].

Requirement: a learned policy that controls the hand and the arm together, with the solution space narrowed to one grasp strategy, the pinch grasp, which the policy has to acquire, hold, and release.

1.2.3 Threading as a Long-Horizon Cyclic Task

A long-horizon cyclic task repeats a fixed sequence of phases many times while the task state advances from one repetition to the next. Threading with a pinch grasp is such a task. One turn is limited by the range of the wrist and the fingers, so the hand has to approach the nut, grasp it, turn it, release it, and return for the next grasp, cycle after cycle. Meanwhile the nut advances along the thread by a fraction of a pitch in every cycle, so no two cycles start from the same state. Many tasks in assembly and everyday manipulation share this structure, for example driving a screw, turning a crank, winding a cable, or wiping a surface.

Three properties make such tasks hard. First, errors compound across cycles. A grasp that lands slightly off the intended faces, a release that disturbs the nut, or a return that brings the fingers back at the wrong height carries over into the next cycle and accumulates. For a learned policy this is worse, because small deviations lead it to states it was not trained on, where its errors grow further [20]. Second, failures concentrate at the transitions between phases, where the contact between hand and nut is made or broken, rather than within a single phase. Third, a single RL policy for the whole task needs a reward that covers every phase and every transition, and it has to assign the outcome of a late cycle to decisions made many steps earlier. Reward shaping over many phases and credit assignment over the full horizon break down for such a policy [12].

The common answer is to decompose the task into skills, train one policy per skill, and chain the policies at deployment [15]. Chaining moves the difficulty to the transitions. The final state of one policy has to lie in the region where the next policy succeeds, which needs extra machinery that fine-tunes the sub-policies for their neighbors [15], and several policies together with a switching logic have to be deployed. A second answer is to imitate complete long-horizon demonstrations with one generative policy [21], [22]. This avoids the reward problem, but it needs demonstrations of the whole task, and teleoperated demonstrations of forceful dexterous manipulation are hard to collect [10].

Requirement: a training signal that is easy to specify phase by phase, and one deployable policy that sustains many cycles without switching between separate policies.

Taken together, the three requirements point to one approach: decompose the task where the training signal is generated, and unify it where the policy is deployed.

1.3 Approach

This thesis decomposes the task during data generation and unifies it at deployment, as Figure 1.4 summarizes. Each part of the approach answers one of the difficulties of Section 1.2. A simplified kinematic thread model couples the axial travel of the nut to its rotation while keeping the finger–nut contacts, which makes massively parallel training with proximal policy optimization (PPO) feasible (Section 1.2.1). A privileged RL teacher, which observes the nut pose from the simulator, learns the pinch-grasp turn: approach, grasp, rotation, and goal reaching (Section 1.2.2). Scripted controllers release the nut and return the hand. Chained with the teacher, they form a hybrid teacher that generates complete multi-cycle demonstrations whose phase boundaries are triggered by physical events. A single visuomotor student is then distilled from these demonstrations by conditional flow matching (CFM) [23]. This teacher–student structure follows earlier work that distills privileged policies into sensor-based students [24], [25]. The student cannot observe the true nut pose, so it replaces the teacher's privileged nut state with perception. It acts from proprioception and a fixed RGB-D camera whose model is calibrated against the physical cell. The depth image is converted into a metric point cloud [26], and the rendering is randomized during data generation [24], so that the student sees similar inputs in simulation and on hardware. Teacher, student, and robot share one Cartesian action path. The student is therefore meant to run the whole cycle on hardware without privileged state, without switching between policies, and without real-world data.

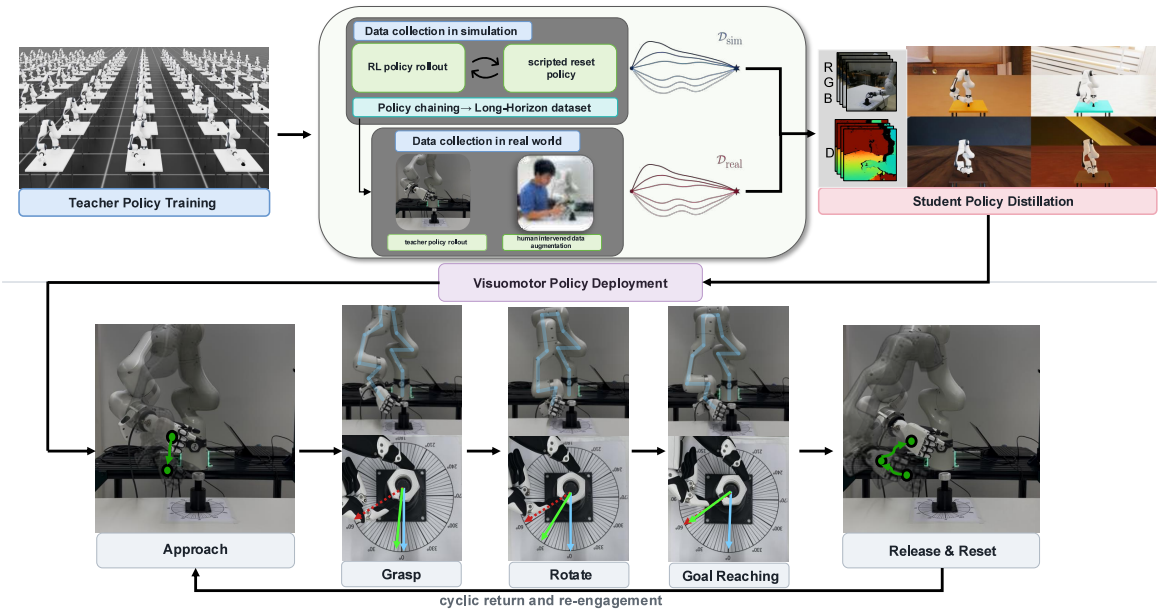


Figure 1.4: Overview of the approach: decompose during data generation, unify at deployment. Top: a privileged RL teacher is trained in parallel simulated workcells. During data collection its rollouts are chained with scripted release-and-reset controllers into long-horizon demonstrations $\mathcal{D}_{\text{sim}}$. A conditional flow-matching student is distilled from them and acts from RGB and depth. Bottom: the phases of one cycle on the physical robot, approach, grasp, rotation, goal reaching, and release-and-reset, after which the hand returns for the next cycle.

1.4 Research Questions

Note: Perhaps this section should be removed from the introduction.

The approach rests on two claims, that one student can replace a chain of policies and that it can be trained on simulated data alone, and on one design choice, the student that performs the unification. Three research questions test them.

RQ1 (long horizon). Does one student distilled from complete multi-cycle demonstrations sustain consecutive threading cycles better than a single-cycle policy that is invoked repeatedly?

RQ2 (student design). Which student design distills the hybrid-teacher demonstrations into a policy that sustains the cyclic task? The ablations designed for it replace the flow-matching action head with deterministic regression on long short-term memory (LSTM) and Transformer backbones, and the point-cloud encoder with an image-space encoder.

RQ3 (sim-to-real). Can a student trained on simulated demonstrations only transfer zero-shot to the physical hand–arm system?

RQ1 tests the answer to the long-horizon difficulty, RQ2 asks which student makes the unification work, and RQ3 tests the answer to the transfer difficulty. The experiments for RQ1 and RQ2 run in simulation (Chapter 4), and those for RQ3 on hardware (Chapter 5). The answer to the dexterity difficulty, the privileged teacher, is tested by the teacher training that Chapter 4 reports before the student experiments.

1.5 Contributions

The thesis makes the following contributions.

1. A formulation of cyclic nut threading as an ordered sequence of phases with explicit cyclic and episodic tracking errors, and a hybrid teacher that combines a privileged RL policy for approach, grasp, rotation, and goal reaching with scripted controllers for release and return. The hybrid teacher generates coherent multi-cycle demonstrations with physically gated phase boundaries.

2. A unified visuomotor student distilled by conditional flow matching from a point-cloud representation of an aligned RGB-D view, proprioception, a global trajectory-progress scalar, and a process-phase encoding. Ablations compare it with a single-cycle student, deterministic regression heads, and an alternative vision encoder. Their results are TBD.

3. A zero-shot sim-to-real recipe for the student: a camera model calibrated against the physical cell, a metric point cloud cropped to the workspace, visual randomization of the rendered demonstrations, and a Cartesian action path shared by teacher, student, and robot.

4. An evaluation of the student in simulation on the hexagonal training nut and on hardware across three nut sizes and four nut profiles, of which only the M24 hexagonal nut appears in the training data. The simulation results are reported in Chapter 4. The hardware results are TBD.

This work was carried out together with the advisor, who developed substantial parts of the hybrid teacher, the distillation stack, and the hardware experiments. The acknowledgments state this attribution.

1.6 Outline

This thesis comprises seven chapters: Introduction, State of the Art, Methodology, Experiments in Simulation, Experiments on Real Hardware, Discussion, and Conclusion and Future Work. This chapter motivated threading as a skill for dexterous robots, named the difficulties of learning it, and stated the approach and the research questions. Chapter 2 reviews the background and related work, Chapter 3 presents the method, Chapters 4 and 5 report the experiments in simulation and on hardware, Chapter 6 interprets the results, and Chapter 7 concludes the thesis. The remainder of this section outlines these chapters in more detail.

Chapter 2 provides the background that the method builds on: the thread geometry that the thread model uses, RL with privileged information, imitation learning and compounding error, generative action heads, and operational-space control. It then reviews related work on robotic assembly and threading, dexterous manipulation, visual representations for manipulation, long-horizon tasks and policy chaining, and teacher–student distillation.

Chapter 3 presents the method, with each part answering one difficulty of Section 1.2: the problem formulation, the simulated system with its thread model and controller, the privileged teacher, the hybrid teacher and the generation of demonstrations, the distillation of the student by conditional flow matching, and the evaluation protocol.

Chapter 4 reports the experiments in simulation. It defines the evaluation metrics and reports the ablations for RQ1 and RQ2.

Chapter 5 describes the hardware setup, the zero-shot transfer protocol, and the real-world evaluation for RQ3 on the training nut, across nut sizes and geometries, and under long-horizon stress.

Chapter 6 interprets the results with respect to the research questions, discusses what they mean for robotic assembly, and states the limitations of the work.

Chapter 7 returns to the assembly motivation of this chapter, summarizes what a dexterous robot can now do in threading and what is still missing, and outlines future work.

Chapter 2

State of the Art

This chapter provides the background that the method of Chapter 3 builds on and places the thesis within related work. Section 2.1 introduces the theoretical concepts in the order in which the method uses them: the geometry of threaded joints that the thread model reproduces, reinforcement learning for the teacher, imitation learning for the distillation, action chunking and generative action heads for the student, and operational-space control for the action path that teacher, student, and robot share. Section 2.2 reviews related work along the three difficulties that threading poses for a robot: it covers robotic assembly and threading, dexterous manipulation with learned policies, visual representations for manipulation, long-horizon tasks and policy chaining, and teacher–student distillation, and it closes with the gap that this thesis addresses.

2.1 Theoretical Background

2.1.1 Threaded Joints and Thread Geometry

Screws are divided into fastening screws and motion screws, and a nut on a bolt is a fastening joint [2]. Threaded fasteners make a joint nonpermanent, so that it can be released again for field assembly, disassembly, maintenance, and adjustment [1]. Their threads are standardized, for example in the ISO metric M series [1], whose geometry Figure 2.1 shows. The thread converts a large circumferential motion into a small axial motion, and a small circumferential force into a large axial force [2]. Assembling a threaded joint and securing it are treated as separate design topics [2]. This thesis addresses the assembly motion, turning the nut along the thread, and not the securing of the joint.

The quantity that links rotation and axial travel is the pitch ρ, the distance between adjacent threads [1]. For a single-start thread, adjacent threads are successive turns of one helix. A nut that turns by an angle $\Delta\theta$ about the bolt axis therefore advances along the axis by $\rho\,\Delta\theta/(2\pi)$, that is, by one pitch per revolution. Because a partial turn advances the nut by only a fraction of a pitch, the axial position at which a hand must grasp the nut changes with every turn.

2.1.2 Reinforcement Learning

In RL, an agent learns a behavior from interaction and a scalar reward, not from demonstrations [12], [27]. At every step an agent observes its environment, chooses an action, and receives a reward that rates the outcome. The agent has to discover the actions that lead to

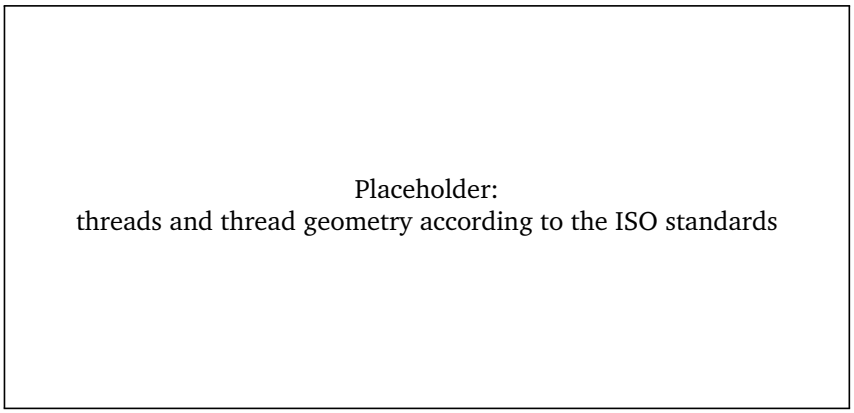


Figure 2.1: Threads and thread geometry according to the ISO standards. Placeholder.

high reward by trying them, and it has to attribute a reward to the earlier actions that made it possible.

The interaction is formalized as a Markov decision process (MDP) $(\mathcal{X}, \mathcal{A}, P, r, \gamma)$ with state space $\mathcal{X}$, action space $\mathcal{A}$, transition kernel $P(\boldsymbol{x}_{t+1} \mid \boldsymbol{x}_t, \boldsymbol{a}_t)$, reward $r(\boldsymbol{x}_t, \boldsymbol{a}_t)$, and discount factor $\gamma \in [0, 1)$ [27]. The Markov property states that the next state depends only on the current state and action, not on the earlier history. A stochastic policy $\pi(\boldsymbol{a} \mid \boldsymbol{x})$ maps states to distributions over actions. Executing it generates a trajectory $\tau = (\boldsymbol{x}_0, \boldsymbol{a}_0, r_0, \boldsymbol{x}_1, \dots)$, and the goal is a policy that maximizes the expected discounted return

$$J(\pi) = \mathbb{E}_{\tau \sim p_\pi}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]. \tag{2.1}$$

The discount factor sets how far ahead the agent plans. A reward that lies t steps in the future is weighted by γ^t, so rewards beyond roughly $1/(1-\gamma)$ steps contribute little.

Value functions measure how good a state or an action is under a given policy. The state value $V^\pi(\boldsymbol{x}) = \mathbb{E}_\pi[\sum_{l \geq 0} \gamma^l r_{t+l} \mid \boldsymbol{x}_t = \boldsymbol{x}]$ is the expected return when the policy starts in $\boldsymbol{x}$. The action value $Q^\pi(\boldsymbol{x}, \boldsymbol{a})$ is the expected return when the agent first takes $\boldsymbol{a}$ and then follows π. Their difference is the advantage

$$A^\pi(\boldsymbol{x}, \boldsymbol{a}) = Q^\pi(\boldsymbol{x}, \boldsymbol{a}) - V^\pi(\boldsymbol{x}), \tag{2.2}$$

which is positive for actions that are better than the policy's average behavior in that state and negative for worse ones. The state value satisfies the Bellman equation $V^\pi(\boldsymbol{x}_t) = \mathbb{E}_\pi[r_t + \gamma V^\pi(\boldsymbol{x}_{t+1})]$, which relates the value of a state to the reward of one step and the value of its successor [27]. This recursion lets a value function be learned from single transitions instead of complete trajectories.

Value functions rate states and actions, but the policy itself can be improved directly by gradient ascent on the expected discounted return J of (2.1). This requires a differentiable policy π_θ, here a neural network with parameters θ, and a gradient of J that can be estimated from sampled trajectories. The likelihood-ratio form of the policy gradient,

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim p_{\pi_\theta}}\left[\sum_t \nabla_\theta \log \pi_\theta(\boldsymbol{a}_t \mid \boldsymbol{x}_t) \Psi_t\right], \tag{2.3}$$

estimates this gradient from sampled trajectories alone, without a model of the transition kernel [27], [28]. The weight Ψ_t scores the action $\boldsymbol{a}_t$. With the return that follows step t as Ψ_t, as in REINFORCE [28], the estimate is unbiased but has high variance, because the return accumulates the randomness of every later action and transition.

Actor–critic methods reduce this variance by learning a value function alongside the policy. The actor is the policy π_θ, and the critic is a value network V_ϕ that is regressed onto the observed returns. The critic replaces the return in Ψ_t by an estimate of the advantage (2.2), that is, of how much better an action turned out than the critic expected. After one step, the critic compares its expectation $V_\phi(\boldsymbol{x}_t)$ with the observed reward plus its estimate of the next state. The difference is the temporal-difference residual $\delta_t = r_t + \gamma V_\phi(\boldsymbol{x}_{t+1}) - V_\phi(\boldsymbol{x}_t)$, which is positive when the step went better than expected. Generalized advantage estimation (GAE) [29] averages these residuals over the following steps with exponentially decaying weights,

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda_{\mathrm{GAE}})^l \, \delta_{t+l}, \tag{2.4}$$

where $\lambda_{\mathrm{GAE}} \in [0, 1]$ trades the bias of the critic against the variance of the observed returns.

The critic is needed only to compute $\hat{A}_t$ during training and is discarded afterwards. It may therefore receive information that the actor will not have at deployment. In an asymmetric actor–critic [30], the critic observes the full, noise-free simulator state, while the actor acts only on the observations available to it. The privileged critic gives more accurate value estimates and therefore better advantages for the actor, but the actor never depends on the privileged state.

Proximal policy optimization (PPO) [31] is an on-policy actor–critic algorithm. On-policy methods estimate the gradient from data collected by the current policy, so every batch becomes outdated as soon as the policy changes. PPO reuses each batch for several epochs of minibatch updates and keeps the updated policy close to the one that collected the data. It maximizes the clipped surrogate objective

$$L^{\mathrm{clip}}(\theta) = \mathbb{E}_t\Big[\min\big(\eta_t(\theta)\hat{A}_t,\ \mathrm{clip}(\eta_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big], \qquad \eta_t(\theta) = \frac{\pi_\theta(\boldsymbol{a}_t \mid \boldsymbol{x}_t)}{\pi_{\theta_{\mathrm{old}}}(\boldsymbol{a}_t \mid \boldsymbol{x}_t)}, \tag{2.5}$$

where $\eta_t(\theta)$ is the probability ratio between the new and the old policy and ϵ is the clipping parameter. Once the ratio leaves the interval $[1-\epsilon, 1+\epsilon]$ in the direction that the advantage favors, the objective stops growing, and the gradient no longer pushes the policy further. The full loss adds a squared-error loss for the critic and an optional entropy bonus that keeps the policy stochastic,

$$L(\theta, \phi) = -L^{\mathrm{clip}}(\theta) + c_V \, \mathbb{E}_t\Big[\big(V_\phi(\boldsymbol{x}_t) - \hat{R}_t\big)^2\Big] - c_H \, \mathbb{E}_t\big[\mathcal{H}\big(\pi_\theta(\cdot \mid \boldsymbol{x}_t)\big)\big], \tag{2.6}$$

with the return target $\hat{R}_t = \hat{A}_t + V_\phi(\boldsymbol{x}_t)$, the critic coefficient c_V, the entropy coefficient c_H, and the policy entropy $\mathcal{H}$ [31]. One training epoch collects a rollout of fixed length from every environment, computes the advantages (2.4), and then runs several passes of minibatch gradient descent on (2.6). On-policy learning needs many samples, and GPU-parallel simulators supply them by stepping thousands of environments at once. On such simulators, RL has trained in-hand reorientation that transferred to hardware [7] and regrasping and throwing with simulated hand–arm systems [13].

The reward is the only specification of the task that the agent receives, and its design largely decides whether learning succeeds [12]. A sparse reward that pays only on success is easy to specify, but in a high-dimensional action space random exploration almost never reaches success, so the agent receives no learning signal. Reward shaping adds dense terms that pay for intermediate progress, such as moving the hand toward the object or turning it toward the goal. Shaping can, however, change which policy is optimal. Ng et al. [32] show that a shaping term of the form $F(\boldsymbol{x}_t, \boldsymbol{x}_{t+1}) = \gamma\Phi(\boldsymbol{x}_{t+1}) - \Phi(\boldsymbol{x}_t)$, with an arbitrary potential function Φ over states, leaves the optimal policy unchanged, and that other shaping terms carry no such guarantee. A term that is not of this form may be exploited. The agent

optimizes the reward as written rather than the task as intended. A term that pays for every reduction of a distance, for example, pays again each time the agent moves away and comes back, so a policy can collect it by oscillating. In simulation the agent can also exploit inaccuracies of the physics, as the interpenetration through the thread flanks of Section 1.2.1 illustrates.

RL thus produces a policy from a reward alone, at the cost of many simulated trials and a carefully shaped reward. When an expert policy or recorded demonstrations are available, a policy can instead be learned by supervised imitation, which the next subsection introduces.

2.1.3 Imitation Learning

Imitation learning (IL) is the problem of learning a policy from the demonstrations of an expert rather than from a reward. IL methods differ in how they use the demonstrations. The simplest method, behavior cloning (BC), reduces IL to supervised learning [20]. It treats every demonstrated pair of observation and expert action as a labeled training example and fits the learned policy, called the student, to these labels. Given a dataset $\mathcal{D} = \{(\boldsymbol{y}_t, \boldsymbol{u}_t^*)\}$, BC solves the regression problem

$$\theta^* = \arg\min_{\theta} \mathbb{E}_{(\boldsymbol{y},\boldsymbol{u}^*)\sim\mathcal{D}}[\ell(\pi_\theta(\boldsymbol{y}), \boldsymbol{u}^*)], \tag{2.7}$$

with a per-sample loss ℓ such as the squared error. BC is trained on the states that the expert visits but evaluated on the states that the student visits. Small prediction errors move the student into states that are not covered by the demonstrations, and the errors compound. Ross et al. [20] show that the expected cost of a BC policy with per-step error ε can grow as $O(T^2\varepsilon)$ over a horizon T. For a cyclic task, small deviations in grasp height or nut angle carry over from one cycle into the next.

Dataset aggregation (DAgger) [20] counters this by querying the expert on the states that the student itself visits. The environment is driven by a mixture that executes the expert's action with probability β and the student's action otherwise, the expert labels every visited state, and the student is retrained on the aggregated dataset. The expected cost then grows only linearly in the horizon. DAgger requires an expert that can be queried at arbitrary states. A policy in simulation satisfies this requirement, a recorded demonstration does not.

2.1.4 Action Chunking

A policy that predicts one action per observation is queried at every control step. An action-chunking policy instead predicts a sequence of H future actions, the action chunk $[\boldsymbol{u}_t, \ldots, \boldsymbol{u}_{t+H-1}]$, from the observation $\boldsymbol{y}_t$ [33]. It executes the first $H_\mathrm{e} \leq H$ actions of the chunk without querying the policy again and then predicts a new chunk from the current observation. The policy thus plans further ahead than it acts, which keeps consecutive actions consistent while the policy still responds to new observations at every query [21]. Chunking has three benefits for imitation learning and one cost.

First, chunking counters the compounding errors of BC (Section 2.1.3). A policy that executes H_e actions per query makes only T/H_e decisions over a task of T steps. The effective horizon of the task shrinks by the factor H_e, and fewer errors can compound [33].

Second, chunking handles demonstrations that are not Markovian in the observation. Human demonstrators pause in the middle of a demonstration, for example during teleoperation, and the observation does not show how long the pause has lasted. A single-step policy learns that it stands still in such a state, but not for how long, and it can get stuck [21], [33].

A chunk covers the pause together with the motion that follows it, so the policy can learn to resume. Similarly, when the demonstrations contain several valid behaviors, a policy that samples every action independently can switch between them from one step to the next and produce jittery motion. A chunk is sampled as a whole and follows one behavior [21].

Third, chunking decouples inference from the control rate. A large policy, such as a vision-language-action model, can need longer for one inference than one control period. A chunk supplies actions for several control periods, so the robot keeps moving while the next chunk is computed [34]. In this way, a vision-language-action model controls robots at up to 50 Hz [22].

The cost of chunking is reactivity. While the actions of a chunk execute, the policy does not observe their effect and cannot respond to a disturbance before its next query. In tasks with uncertain contact, a long execution horizon therefore trades robustness against compounding errors for responsiveness to the contact. Two remedies keep the chunk prediction and restore feedback. Temporal ensembling queries the policy at every step and executes an exponentially weighted average of all predictions for the current step from the chunks that are still valid [33]. Real-time chunking generates the next chunk while the current one executes. It fixes the actions that will execute before the new chunk is ready and fills in the rest of the new chunk consistently with them, which avoids discontinuities at the switch between chunks [34]. Predicting a chunk requires a model of the distribution over action sequences, which the generative action heads of the next subsection provide.

2.1.5 Generative Action Heads: Diffusion and Flow Matching

A generative model learns a probability distribution from a set of samples and produces new samples from it. Diffusion models [35] do so by transforming Gaussian noise step by step into a data sample with a learned network. Conditioned on a text prompt, latent diffusion models generate high-resolution images that match the prompt [36].

In robot learning, the same principle generates actions instead of images. A generative action head learns the conditional distribution $p(\boldsymbol{u} \mid \boldsymbol{y})$ of the demonstrated actions given the observation and draws an action from it at run time. A regression head instead predicts a single action per observation, and the Gaussian policy of Section 2.1.2 represents only a single mode. The generative action head has two advantages. First, when the demonstrations contain several valid actions for the same observation, a sample takes one of them rather than their average [21]. The average of two valid actions need not be valid itself, for example: if an obstacle can be passed on either side, the average drives into it. Second, the head can generate a whole action chunk (Section 2.1.4) as one sample, so that all actions of the chunk follow the same mode [21]. Generative action heads are also used in large generalist policies, such as the vision-language-action model π_0, which is trained on data from many robots and tasks [22]. Two families of generative action heads are common in robot learning, diffusion and flow matching. Both start from Gaussian noise and transform it step by step into an action, but they learn this transformation differently.

Diffusion Policy [21] learns to remove noise. During training, Gaussian noise is added to a demonstrated action, and a network that is conditioned on the observation predicts the added noise. The magnitude of the noise depends on a randomly drawn iteration index through the noise schedule, a hand-designed sequence of noise variances that fixes how much noise each iteration adds or removes. At run time, sampling starts from pure noise and applies the network over a fixed number of denoising iterations, each of which removes part of the predicted noise. The number of iterations sets the inference cost. Diffusion Policy predicts action chunks and executes several actions of a chunk before it predicts again.

Flow matching [23], [37] learns the transformation from noise to action as a velocity

field. Each noise sample can be pictured as a particle that moves during a flow time λ from 0 to 1. A network $\boldsymbol{v}_\theta(\boldsymbol{u}, \lambda \mid \boldsymbol{y})$ predicts the velocity of the particle at position $\boldsymbol{u}$ and time λ, and integrating this velocity from a noise sample $\boldsymbol{u}^0$ at $\lambda = 0$ yields an action $\boldsymbol{u}^1$ at $\lambda = 1$. The velocity field that maps the noise distribution onto the demonstrated actions is unknown, but a training target can be built from pairs of a noise sample and a demonstrated action. The simplest construction connects the two by a straight line,

$$\boldsymbol{u}^\lambda = (1-\lambda)\boldsymbol{u}^0 + \lambda \boldsymbol{u}^1, \qquad \boldsymbol{u}^0 \sim \mathcal{N}(\mathbf{0}, I), \tag{2.8}$$

along which the velocity is the constant $\boldsymbol{u}^1 - \boldsymbol{u}^0$. Training regresses the network onto this velocity at random points of random lines,

$$L(\theta) = \mathbb{E}_{(\boldsymbol{y},\boldsymbol{u}^1)\sim\mathcal{D},\ \boldsymbol{u}^0\sim\mathcal{N}(\mathbf{0},I),\ \lambda\sim U[0,1]} \left\| \boldsymbol{v}_\theta(\boldsymbol{u}^\lambda, \lambda \mid \boldsymbol{y}) - \left(\boldsymbol{u}^1 - \boldsymbol{u}^0\right) \right\|_2^2. \tag{2.9}$$

Many lines pass through the same point, so the network learns the average of their velocities. Lipman et al. [23] show that this average is exactly the unknown velocity field, so the simple loss (2.9) trains the correct model. At run time, an action is generated by drawing $\boldsymbol{u}^0 \sim \mathcal{N}(\mathbf{0}, I)$ and following the learned field with a few explicit Euler steps $\boldsymbol{u}^{\lambda+\Delta\lambda} = \boldsymbol{u}^\lambda + \Delta\lambda\, \boldsymbol{v}_\theta(\boldsymbol{u}^\lambda, \lambda \mid \boldsymbol{y})$.

Diffusion and flow matching are closely related. A diffusion model corresponds to flow matching with a particular Gaussian path between noise and data, whose trajectories are curved [23]. Figure 2.2 contrasts the two. The straight-line path of (2.8) gives flow matching three advantages. First, training is simpler. The regression target is the difference of two samples, and the path fixes how noise and data are mixed over the flow time, so no noise schedule has to be designed. Second, sampling is faster. A straight path can be followed with a few large integration steps, whereas a curved path needs many small ones. On image generation, flow matching reaches the sample quality of diffusion with fewer network evaluations [23]. Third, sampling is the deterministic integration of an ordinary differential equation (ODE) from a given noise sample. The number of integration steps can therefore be chosen at run time, which trades accuracy against inference time without retraining. Fast sampling matters for robot control, because a new action chunk must be ready within the control period. π_0 uses flow matching to generate action chunks for control at up to 50 Hz [22].

In a policy, the flow-matching action head sits on top of a backbone. The backbone encodes the observation, for example camera images, the robot state, or a language instruction, into a condition vector $\boldsymbol{c}$, which takes the place of $\boldsymbol{y}$ in (2.9). The head is the velocity network $\boldsymbol{v}_\theta(\boldsymbol{u}^\lambda, \lambda \mid \boldsymbol{c})$. It receives the noisy action or action chunk $\boldsymbol{u}^\lambda$, the flow time λ, and the condition, and it outputs a velocity of the same dimension as the action. The flow time is a scalar and is usually embedded with sinusoidal features before it enters the network [22]. The velocity network is typically an multilayer perceptron (MLP) or a Transformer. The condition enters it by concatenation with the input, by a feature-wise scaling and shifting of the hidden layers [21], or as additional tokens that the action tokens attend to [22]. The backbone is evaluated once per generated action chunk, the head once per integration step. Each additional step therefore costs only one evaluation of the head. π_0 follows this structure: a second set of Transformer weights, called the action expert, generates action chunks by flow matching and attends to the features of a pre-trained vision-language model [22]. Section 3.5.2 describes the backbone and the head of the student in this thesis.

2.1.6 Operational-Space Control

A policy controls a robot arm through its action space, and the choice of this space decides how much of the motion the policy has to learn itself. The most direct choice is the joint

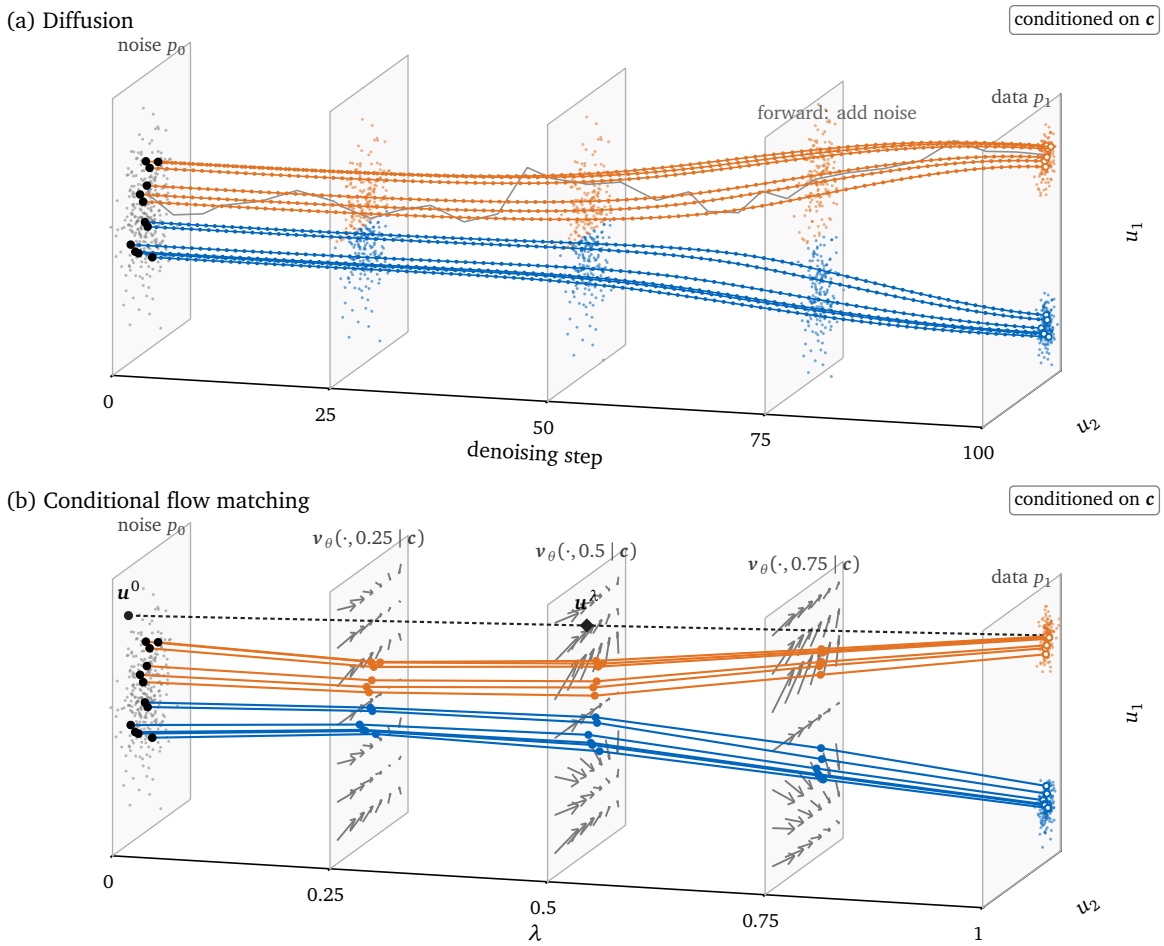


Figure 2.2: Diffusion and conditional flow matching for a bimodal action distribution in a two-dimensional toy action space (u_1, u_2). Generation time runs from left to right, from Gaussian noise p_0 to the demonstrated actions p_1, and each slice shows the action space at one point in time. (a) A diffusion model adds noise to demonstrated actions (gray path) and learns to reverse this process over many denoising steps along curved trajectories. The point clouds on the slices are intermediate denoising samples, colored by the mode they reach. (b) Conditional flow matching regresses a velocity field onto the straight path between a Gaussian sample and a data sample (dashed). Integrating the learned field (gray arrows on the slices at $\lambda = 0.25$, 0.5, and 0.75) from noise reaches either mode in a few ODE steps.

space. On a seven-DoF arm, the policy outputs seven joint position targets at every step, and a joint position controller tracks them in simulation and on the hardware. The policy then learns the complete motion of the arm, including the coordination of the joints that moves the end effector to where the task needs it. Joint-space actions impose no structure on this motion and leave the policy free to explore. Grasping pipelines that train arm and hand together with RL use them. For example, a humanoid with two seven-DoF arms learns to grasp, lift, and hand over objects with joint position targets for its arm and hand joints [38].

The alternative moves part of the motion into a controller that runs both during training and at deployment. The policy then commands a target in the operational space, the space in which the task is described [39]. For assembly, this is the six-dimensional pose of the end effector, its position and orientation in Cartesian space, and the controller computes the joint torques that move the end effector toward the commanded pose. The action space shrinks from the number of joints to six dimensions, and it is expressed in the coordinates of the task, for example relative to a bolt. With an impedance controller in end-effector space, RL learns

contact-rich tasks with fewer samples than with joint-space actions [40].

Impedance is the second reason for this choice. A stiff position controller drives through an unexpected contact and can produce forces that damage the robot or the parts. An impedance controller instead makes the end effector behave as if it were attached to the target by a spring and a damper [41]. In steady contact, the force on the environment equals the stiffness times the deviation of the end effector from its target, so a small target offset produces only a small force. This protects the robot and the environment. It also gives the policy a way to apply force: a target that lies a small distance behind a contact surface presses the end effector against the surface with the stiffness times that distance. The policy thus regulates contact forces by placing targets, without commanding forces directly.

Task-space motion control with torque inputs and impedance control are treated in standard references [41], [42], [43]. Impedance control is simplest in joint space. Assuming that gravity is compensated, the law

$$\boldsymbol{\tau} = K_q\,(\boldsymbol{q}_{\mathrm{ref}} - \boldsymbol{q}) - D_q\dot{\boldsymbol{q}} \tag{2.10}$$

makes every joint behave like a spring with stiffness K_q and a damper with damping D_q around its reference $\boldsymbol{q}_{\mathrm{ref}}$, where $\boldsymbol{q}$ are the joint positions (Figure a). The spring then acts on the joints, not on the end effector, and the stiffness felt at the end effector changes with the arm configuration. Operational-space impedance control instead defines the spring and the damper directly in the six-dimensional task space and maps them to the joints with the Jacobian. Joint space and operational space are related by the forward kinematics

$$\boldsymbol{x} = \boldsymbol{f}(\boldsymbol{q}), \tag{2.11}$$

which gives the pose $\boldsymbol{x}$ of a frame on the robot for the joint positions $\boldsymbol{q}$. The Jacobian is the derivative of this map, $J(\boldsymbol{q}) = \partial \boldsymbol{f}/\partial \boldsymbol{q} \in \mathbb{R}^{6\times 7}$. By the chain rule, it relates the velocities in the two spaces,

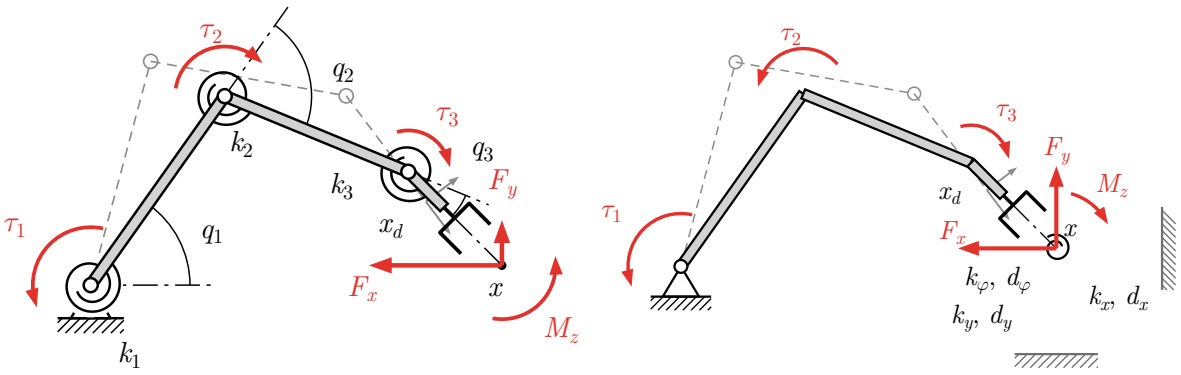
$$\dot{\boldsymbol{x}} = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{q}}\,\frac{\mathrm{d}\boldsymbol{q}}{\mathrm{d}t} = J(\boldsymbol{q})\,\dot{\boldsymbol{q}}. \tag{2.12}$$

The relation is linear in the joint velocities, but the Jacobian itself changes with the configuration. For the orientation, the time derivative of a parameterization such as Euler angles is not the angular velocity of the frame. Controllers therefore use the geometric Jacobian, which maps the joint velocities to the twist $\dot{\boldsymbol{x}}$ of the frame, its linear and angular velocity [42]. By the principle of virtual work, its transpose maps a wrench $\boldsymbol{F}$ at the frame to the joint torques that produce it, $\boldsymbol{\tau} = J^{\mathrm{T}}\boldsymbol{F}$ [42]. The frame can be any point on the robot, for example the point between the fingertips at which the hand holds the nut. A virtual spring and damper at this frame (Figure b), $\boldsymbol{F} = K_p\boldsymbol{e} - K_d\dot{\boldsymbol{x}}$ with the position and axis-angle orientation error $\boldsymbol{e} \in \mathbb{R}^6$ between target and current pose, therefore translates directly into joint torques, and the controller computes them at every step from the current Jacobian. On the seven-DoF arm, the six task dimensions leave one redundant degree of freedom, which a second term controls. The complete law, in the form that the thesis uses, is

$$\boldsymbol{\tau} = J^{\mathrm{T}}\big(K_p\boldsymbol{e} - K_d\dot{\boldsymbol{x}}\big) + N^{\mathrm{T}}M\big(K_{p,0}(\boldsymbol{q}_0 - \boldsymbol{q}) - K_{d,0}\dot{\boldsymbol{q}}\big), \tag{2.13}$$

where K_p and $K_d = 2\sqrt{K_p}$ are diagonal stiffness and critical-damping gains, and the second term pulls the redundant degree of freedom toward a rest posture $\boldsymbol{q}_0$ through a nullspace projector N with posture gains $K_{p,0}$ and $K_{d,0}$. The joint-space inertia M converts the posture term from a joint acceleration into a torque.

The nullspace projector decides how strongly the posture term interferes with the task. It is dynamically consistent when $N^{\mathrm{T}} = I - J^{\mathrm{T}}\bar{J}^{\mathrm{T}}$ with the inertia-weighted pseudoinverse



(a) Joint-space impedance. **(b)** Operational-space impedance.

Figure 2.3: Impedance control, illustrated on a planar arm with three joints. (a) A virtual spring and damper with stiffness k_1, k_2, k_3, the diagonal entries of K_q, act directly on each joint, pulling it toward its reference q_{ref}. (b) A virtual spring and damper with stiffness k_x, k_y, k_φ and damping d_x, d_y, d_φ, the diagonal entries of K_p and K_d, act on the controlled end-effector frame along each task direction instead. The resulting wrench (F_x, F_y, M_z) is mapped to the joint torques τ_1, τ_2, τ_3 through J^{T}.

$\bar{J} = M^{-1}J^{\mathrm{T}}(JM^{-1}J^{\mathrm{T}})^{-1}$ and joint-space inertia M [44], [45]. Torques in the nullspace then produce no acceleration of the controlled frame. A projector built from a pseudoinverse of the Jacobian without inertia weighting needs no inertia model, but it decouples the nullspace from the task only statically, so that posture torques can still disturb the task during motion [44]. The controllers in simulation and on the physical robot both use the dynamically consistent projector (Section 5.1). Equation (2.13) is the Factory and FORGE controller [14], [16]. Scaling the spring and damper term by the task-space inertia $\Lambda = (JM^{-1}J^{\mathrm{T}})^{-1}$ and compensating the nonlinear dynamics yields the full operational-space formulation, in which the closed loop becomes a decoupled unit-mass system in every task direction [42].

2.2 Related Work

The related work is ordered along the difficulties of threading. Section 2.2.1 reviews robotic assembly and threading, the application itself. Section 2.2.2 reviews learned dexterous manipulation (difficulty (b)), Section 2.2.3 reviews visual representations for manipulation, and Sections 2.2.4 and 2.2.5 review long-horizon tasks and the teacher–student distillation with which this thesis addresses them (difficulty (c)). Section 2.2.6 states the gap.

2.2.1 Robotic Assembly and Threading

Robotic assembly, for example peg-in-hole insertion, gear meshing, connector mating, and nut–bolt fastening, requires contact-rich interaction with high precision and accuracy [17]. In mass production, dedicated fixtures position the parts precisely and thereby reduce the uncertainty of the hole pose [46]. Learning-based methods train assembly policies in simulation. Factory [14] combines signed-distance-field collision detection, contact reduction, and a Gauss–Seidel solver to simulate 1000 nut-and-bolt assemblies in real time on a single GPU, and it learns nut-and-bolt assembly with RL through separate pick, place, and screw subpolicies. FORGE [16] trains force-guided exploration under pose uncertainty and demonstrates on a real robot a multistage planetary-gear assembly that includes nut threading, and

IndustReal [17] transfers simulation-trained assembly policies to a physical arm. All three use a parallel-jaw gripper.

Screwdriving with a manual screwdriver has been addressed with analytical models and force control, first with the tool rigidly attached to a robot arm [8] and then with the tool held by a Shadow Hand [9]. For an industrial screwdriving tool on an impedance-controlled arm, a physics-informed learned model describes the motion of the screw tip [46]. With multi-fingered hands, twisting the lids off bottle-like objects has been learned with two hands by sim-to-real RL [47], and unscrewing a bottle lid with one hand has been learned from teleoperated demonstrations with a visuomotor diffusion policy [48]. Play2Perfect [19] pretrains a goal-conditioned dexterous manipulation policy through play and fine-tunes it for precise assembly, including screwing a furniture leg into a table fixture. It thus covers goal-directed screwing of a furniture leg, but not of small fasteners such as nuts. The only reviewed work that fastens nuts with a dexterous hand is DexScrew [18]. It mounts a 12-DoF hand on a robot arm and replaces the thread geometry by a revolute-joint proxy to learn a rotational finger gait with RL. The gait assists teleoperation and supports the collection of real demonstrations for tactile behavior cloning. The RL policy is rewarded for the angular velocity of the fastener rather than for reaching a target angle, its episodes end on failure or after a fixed time limit rather than on reaching a goal, and the nut is already installed on the bolt. Goal-directed threading over many grasp-and-release cycles, which this thesis addresses, is left open by these works.

2.2.2 Dexterous Manipulation with Learned Policies

A multi-fingered hand is a general-purpose alternative to task-specific tooling, but it brings high-dimensional control and complex contact dynamics with it [4], [5]. According to a recent survey, analytical models of motion and contact have failed to provide practical solutions for in-hand manipulation because they cannot cope with complex environments and uncertainty, and research has therefore shifted to learning [5]. Model-free RL in simulation learns in-hand reorientation of more than two thousand objects [6]. DeXtreme [7] transfers agile in-hand reorientation from simulation to a comparatively affordable Allegro hand with a vision-based pose estimator, and DexPBT [13] learns regrasping and grasp-and-throw for simulated hand–arm systems with population-based training. DextrAH-G [25] and DextrAH-RGB [24] grasp a wide range of objects with a hand–arm system from depth and RGB images, respectively, and SimToolReal [10] manipulates tool-like objects to random goal poses and generalizes zero-shot to real tools. Without learning, an arm-driven hand has executed a finger gait on a tool whose tip pivots on a supporting plane, using hybrid position/impedance control with reference finger forces [49].

Several works name the high-dimensional action space of the hand or of the hand–arm system as a key challenge of dexterous manipulation [6], [15], [25]. Most of the works above learn one skill, such as grasping, reorienting an object, or moving a tool to goal poses. In-hand reorientation chains goals and counts consecutive successes [7], but the object stays in the hand. Threading additionally requires the hand to release the part and acquire it again while the bolt constrains it, and the grasp must hold under the turning load in between. This thesis narrows the solution space to one grasp strategy, a thumb–index pinch, and lets the learned policy control this pinch together with the arm (Section 3.2).

2.2.3 Visual Representations for Manipulation

A visuomotor policy must extract the task-relevant geometry from camera data, in threading the pose of the nut relative to the hand. Two families of encoders, and combinations of them, are used for this. Image-space encoders apply a convolutional backbone such as ResNet-18 [50] to the RGB image, possibly with depth or other per-pixel channels added, and pool the feature map into a vector. They can start from ImageNet-pretrained weights, but from RGB alone they have to infer metric geometry from appearance, and when they are trained on rendered images they must be made robust to texture and lighting, for example by visual domain randomization [51]. Point-cloud encoders back-project the depth image through the calibrated camera model into a set of 3D points, optionally attach the pixel color, and encode the set with a shared point network followed by global max pooling in the spirit of PointNet [52]. The 3D Diffusion Policy (DP3) encoder [26] is such a compact encoder, a three-layer MLP with max pooling and a projection head, and it uses the point coordinates without color. In real-robot tests with one trial per condition, DP3 generalizes to unseen object colors and slightly changed camera views on which image- and depth-image-based Diffusion Policy [21] fail. The authors attribute the robustness to appearance to the omission of color and the spatial generalization to the 3D representation [26]. Adapt3R [53] combines the two families. Dense 2D features from a pretrained backbone are localized in 3D by the depth map and expressed in the end-effector frame before attention pooling. In simulation, this lets a policy transfer zero-shot to unseen robot arms and camera viewpoints, although its success rate on unseen arms is lower than on the training arm. The student of this thesis uses a point-cloud encoder in the style of DP3 whose points carry their pixel color, and Chapter 4 sets up its comparison with an image-space encoder and with an Adapt3R-style encoder.

2.2.4 Long-Horizon Tasks and Policy Chaining

A long-horizon task can be learned by one policy over the whole horizon or decomposed into skills, and each route has its own difficulties. One policy trained with RL needs a reward that covers every phase, and it has to assign the outcome of a late phase to decisions made many steps earlier. AlphaNIST [54] shows in simulation that a sparse binary task reward can suffice. It trains one policy from scratch on the 20 assembly tasks of the NIST Assembly Task Board #1, which require precise, contact-rich control of small, tight-fitting parts, and reaches 98.2 % overall training success without demonstrations, progress shaping, or scripted primitives. The same framework trains policies that complete ten assembly pairs in one episode without a prescribed order, rewarded only when all ten pairs are complete. This thesis takes a different route. It applies RL only to the pinch-grasp turn, whose reward covers a single cycle (Section 3.3), and reaches the long horizon by imitation.

Imitation avoids the reward problem but moves the difficulty to the data. One policy that imitates the whole task needs demonstrations of the whole task, and teleoperated demonstrations of forceful dexterous manipulation are hard to collect [10].

Decomposing the task into skills shortens the horizon that each skill covers, but it creates difficulties of its own. If one network learns the skills one after another, fine-tuning it on later skills can erase what it learned on earlier ones. This catastrophic forgetting affects supervised and reinforcement learning alike [55]. Separate policies per skill avoid forgetting, but chaining them introduces a transition problem. The states in which one skill terminates may lie outside the initial-state distribution on which the next skill was trained, and the next skill can then fail [56]. Adversarial skill chaining [56] regularizes the terminal-state distribution of each skill toward the initial-state distribution of its successor. Sequential Dexterity [15] learns a transition-feasibility function that fine-tunes each sub-policy toward

states from which its successors succeed, and it uses the same function to switch between sub-policies autonomously, which lets it recover from failures and skip redundant stages. A cyclic task sharpens the transition problem. The same boundaries recur in every cycle while the task state advances, so a task of many cycles crosses each boundary many times, and an error at one crossing carries into the following cycle.

Generative action heads make it practical for one imitation policy to cover a multi-stage task. Such a head samples one of several demonstrated actions for similar observations instead of their average (Section 2.1.5). The flow-matching model π_0 [22] is trained on data from several dexterous robot platforms and applied to multi-stage tasks such as laundry folding, table cleaning, and box assembly. Its demonstrations are collected by teleoperation [22], which is the kind of data that is hard to obtain for forceful dexterous manipulation. Section 2.2.5 returns to these action heads.

The method of this thesis decomposes the task into separately controlled phases only during data generation. The privileged RL teacher executes the pinch-grasp turn, scripted controllers execute release and return, and together they generate complete multi-cycle demonstrations in simulation without teleoperation (Section 3.4). One visuomotor student with a flow-matching head is distilled from these demonstrations and deployed. It learns all phases in one training run, so no phase is fine-tuned after another and catastrophic forgetting does not arise. The phase enters the student only as a conditioning signal derived from physical events, not as a switch between networks, and the student predicts the transition actions itself (Section 3.5.2). The transitions are thus part of what the student learns rather than a problem to be solved between separate policies. RQ1 tests whether this unified student sustains consecutive cycles better than a single-cycle policy that is invoked repeatedly.

2.2.5 Teacher–Student and Visuomotor Distillation

Learning by Cheating [57] trains a privileged agent on ground-truth simulator state by imitating an expert and then uses it to supervise a sensorimotor agent that acts from a forward-facing camera image, the vehicle speed, and a high-level command. In dexterous manipulation, DextrAH-G [25] and DextrAH-RGB [24] distill a privileged RL grasping teacher into a depth-based and an RGB-based student, respectively, with online DAgger. UniGraspTransformer [58] distills 3200 object-specific RL teachers into one Transformer student, and Lin et al. [38] train separate per-object RL specialists on a humanoid and distill their successful rollouts offline into one generalist policy per task that acts from egocentric depth, the object position, and the joint states.

Two further design choices of visuomotor policies matter for this thesis, action chunking and the generative action head. Action chunking (Section 2.1.4) was introduced to imitation learning by Action Chunking with Transformers (ACT) [33], which learns fine-grained bimanual tasks on low-cost arms from teleoperated demonstrations. ACT predicts action chunks with a Transformer-based conditional variational autoencoder and averages the overlapping predictions for the current step with exponential weights, which it calls temporal ensembling. Diffusion Policy [21] predicts action sequences with a generative head and executes a fixed number of their actions before it replans. Its authors attribute to sequence prediction that the policy commits to one mode of multimodal demonstrations and does not get stuck on pauses in them. Chunking has since become a standard component of generalist policies. The vision-language-action model π_0 [22] generates chunks of 50 actions with ten flow-matching integration steps and controls robots at up to 50 Hz. It executes the first 16 or 25 actions of each chunk open loop, because temporal ensembling reduced its performance in early experiments [22]. Executing chunks without pauses for inference has become a research topic of its own. Real-time chunking generates the next chunk while the current one

executes and constrains it to the actions that are already committed [34].

A generative model of the action distribution can represent multimodal demonstrated behavior, for which explicit policies trained by direct regression are not suitable [21]. Diffusion Policy [21] models multimodal action sequences by iterative denoising, and DP3 [26] conditions it on compact point-cloud features and the robot pose. Flow matching [23] regresses a continuous vector field that transports noise to data, and samples are generated by integrating an ODE. The student of this thesis combines a point-cloud encoder, action chunking with temporal ensembling, and a flow-matching head. The generative head suits its training data, which mix nominal teacher actions with the teacher's recoveries from disturbances.

2.2.6 Summary and Research Gap

The related work addresses each difficulty of threading separately. Simulation-based assembly learning makes thread contact tractable [14] and transfers insertion and nut threading to hardware [16], [17], but with a parallel-jaw gripper, a part that is already held when contact begins, and no regrasping during threading [14], [16]. Learned dexterous screwing covers a spinning gait for an installed nut [18] and goal-conditioned screwing of a furniture leg [19]. Policy chaining reaches long horizons at the cost of transitions between separately trained policies [15], [56], and teacher–student distillation yields deployable visuomotor policies for single skills [24], [25]. None of the reviewed works threads a nut over many grasp-and-release cycles with a dexterous hand. Chapter 3 addresses this gap by combining the concepts of Section 2.1: a simplified thread model makes RL in simulation feasible, a privileged teacher learns the pinch-grasp turn, scripted transitions chain it into multi-cycle demonstrations, and one flow-matching student distills these demonstrations for zero-shot deployment.

Chapter 3

Methodology

This chapter presents the method. Section 3.1 formalizes cyclic nut threading. Section 3.2 describes the simulated and physical system: the hand–arm platform, the simplified thread model that makes learning in simulation tractable, and the action contract with its controller. Section 3.3 presents the privileged reinforcement-learning teacher that learns to grasp and turn the nut with the dexterous hand. Section 3.4 explains how the hybrid teacher chains this skill with scripted transitions into complete multi-cycle demonstrations, and how the visual observations of these demonstrations are collected and randomized. Section 3.5 develops the central contribution, the distillation of these demonstrations into a single visuomotor student by conditional flow matching. Figure 3.1 gives an overview.

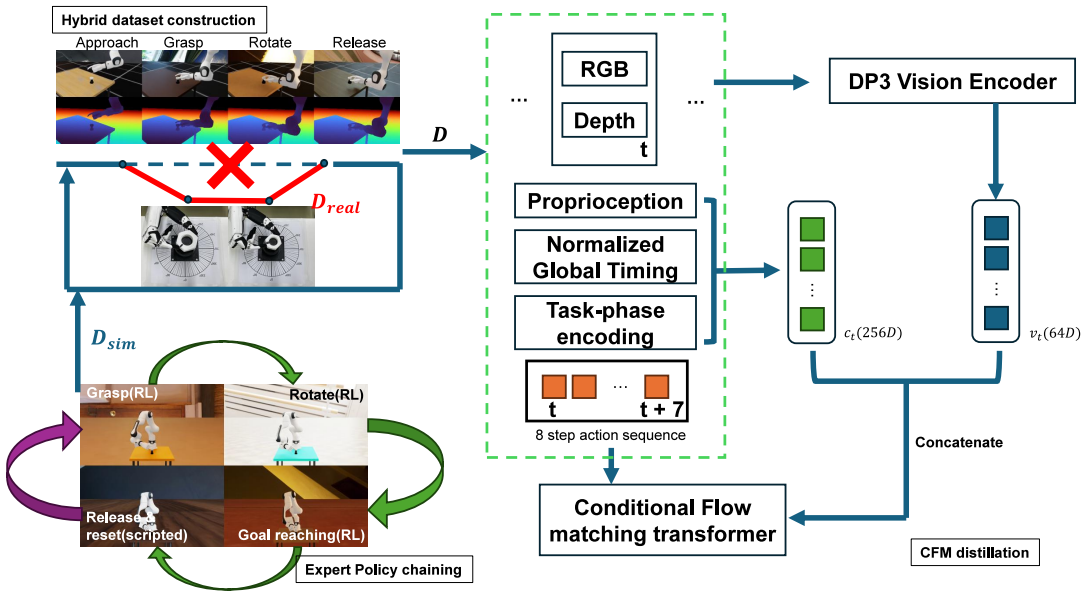


Figure 3.1: The pipeline in detail. Left: the privileged RL policy executes grasp, rotation, and goal reaching and a scripted controller executes release and reset. Chained in a loop, they form the hybrid teacher that generates the simulated demonstrations $\mathcal{D}_{\text{sim}}$. Right: the student encodes the aligned RGB-D observation into a 64-D point-cloud feature, fuses it with proprioception, the normalized global progress, and the process-phase encoding into a 256-D condition, and a conditional flow-matching Transformer predicts an eight-step action chunk.

3.1 Problem Formulation

Cyclic nut threading is decomposed into the ordered phase sequence

$$\mathcal{S} = \big(\underbrace{\text{approach, grasp, rotate, goal reaching}}_{\mathcal{M}_{\mathrm{RL}}:\,\pi_{\mathrm{T}}},\ \underbrace{\text{release, return and reset}}_{\mathcal{M}_{\mathrm{script}}:\,\kappa}\big). \tag{3.1}$$

The active phase at control step t is $m_t \in \mathcal{M} = \mathcal{M}_{\mathrm{RL}} \cup \mathcal{M}_{\mathrm{script}}$. The four phases in $\mathcal{M}_{\mathrm{RL}}$ are executed by a learned teacher policy π_{T}. The two transition phases in $\mathcal{M}_{\mathrm{script}}$ are executed by scripted controllers κ_m. The split is a property of the data-generation stage only: the deployed student receives neither the phase label as a switch nor separate networks per phase.

Let K be the required number of cycles, $\Delta\varphi^*$ the desired clockwise rotation per cycle, and φ_k the unwrapped nut angle about the bolt axis after the goal-reaching phase of cycle k, with φ_0 the angle at the start of the trial. The cyclic tracking error, the episodic tracking error, and the tracking cost of a trajectory τ are defined as

$$e_k = \varphi_k - \varphi_{k-1} - \Delta\varphi^*, \qquad e_{\mathrm{ep}} = \varphi_K - \varphi_0 - K\Delta\varphi^* = \sum_{k=1}^{K} e_k, \qquad E(\tau) = \frac{1}{K}\sum_{k=1}^{K} e_k^2 + \beta_E\, e_{\mathrm{ep}}^2. \tag{3.2}$$

Positive errors indicate overshoot and negative errors lag. The cost squares them so that overshoot in one cycle cannot cancel lag in another, and the weight β_E trades accumulated drift against cycle-wise accuracy. Let $C(\tau) \in \{0, \ldots, K\}$ count the cycles completed before failure or timeout and let $\Pi_{\mathcal{S}}$ denote the policies that execute the phases of $\mathcal{S}$ in order in every cycle. The research problem is to find the policy that chains all phases with maximum completion success and, among those, minimum tracking cost:

$$\Pi_{\mathrm{succ}} = \arg\max_{\pi \in \Pi_{\mathcal{S}}} \Pr_{\tau \sim p_\pi}[C(\tau) = K], \qquad \pi^* = \arg\min_{\pi \in \Pi_{\mathrm{succ}}} \mathbb{E}_{\tau \sim p_\pi}[E(\tau) \mid C(\tau) = K]. \tag{3.3}$$

Unless a study states otherwise, $K = 10$ and $\Delta\varphi^* = 60°$, which corresponds to one face of a hexagonal nut per cycle. The policy π^* is learned from fixed hybrid-teacher demonstrations rather than by optimizing (3.3) on hardware, because on-robot optimization of a contact-rich cyclic task is neither safe nor sample-efficient.

The task is partially observable at deployment, so it is a partially observable Markov decision process (POMDP): the MDP of Section 2.1.2 is extended by an observation model $O(\boldsymbol{z}_t \mid \boldsymbol{x}_t)$, and the policy acts on observations $\boldsymbol{z}_t$ instead of states [27]. The simulator state $\boldsymbol{x}_t$ contains the nut pose, the contact state, and the controller parameters. The student observes only an aligned RGB-D image, its proprioception, and two low-dimensional conditioning signals introduced in Section 3.5.2. A single observation is then no longer sufficient to act optimally, because the information that disambiguates the state may lie in earlier observations. The teacher–student structure of the method separates the two difficulties of this POMDP: the teacher solves the control problem with privileged state, and the student learns to perceive and to reproduce the teacher's behavior from sensor data.

3.2 Simulated and Physical System

The method is trained in simulation on a model of the physical hand–arm system and deployed on that system. This section describes the platform, the thread model that makes learning in simulation tractable, and the action contract that teacher, student, and robot share.

3.2.1 Platform

The robot is a seven-DoF torque-controlled Franka Research 3 (FR3) arm with a five-fingered Inspire RH56DFX right hand mounted on its flange (Figure 3.2). The hand has twelve finger joints driven by six motors. The non-thumb fingers each have one actuated flexion joint with a mechanically coupled distal joint, and the thumb has an abduction joint and a coupled flexion chain. Only three hand degrees of freedom are commanded by the policy: thumb abduction, thumb flexion, and index flexion, which together form the pinch used for threading. The remaining fingers hold a curled posture. This restriction narrows the solution space of the dexterous hand to one grasp strategy, as difficulty (b) of Section 1.2 requires. The nut is threaded onto a bolt that stands on a bench in front of the robot. A fixed RGB-D camera provides the observation. Its extrinsic calibration relative to the robot base was measured on the physical cell and is reproduced in simulation.



(a) FR3 arm. **(b)** Inspire RH56DFX right hand.

Figure 3.2: The robot platform, rendered from the simulation models used in this work. In the assembled system the hand is mounted on the arm flange.

Both teacher and student act at 15 Hz, while the low-level controller closes its loop at the physics rate. Simulator and timing settings are listed in Table A.1 in the appendix.

3.2.2 Simplified Thread Simulation

The thread model is the first answer to difficulty (a) of Section 1.2: the skill is learned in simulation, so the simulated thread has to behave like a mating thread in each of the hundreds of parallel environments that PPO needs. The default Isaac Sim collision pipeline does not provide this: with signed distance field (SDF) mesh colliders on nut and bolt, the simulated thread contact is not accurate enough to reproduce a mating thread. A kinematic thread model also gives the policy a well-posed learning signal: with SDF thread colliders, axial progress of the nut can be obtained by pressing it along the bolt axis, whereas a kinematic coupling makes axial progress a consequence of rotation alone, so that progress can only be earned by turning. The thread model used for teacher training therefore replaces the thread contact by a kinematic coupling, illustrated in Figure 3.3. The nut is attached to the bolt by a passive revolute joint about the bolt axis and a coaxial, position-controlled prismatic joint. With prismatic coordinate d_t, unwrapped and direction-corrected revolute coordinate

θ_t, thread pitch ρ, and the index b_k of the step at which the current turn started, the prismatic target is

$$d_t^* - d_{b_k} = -\frac{\rho}{2\pi}\left(\theta_t - \theta_{b_k}\right), \tag{3.4}$$

so that one revolution produces one pitch of axial travel. The prismatic joint tracks d_t^* with a stiff drive, whose finite stiffness permits small deviations from the ideal helix while the revolute–prismatic pair constrains lateral motion and keeps the nut near the commanded path. The finger–nut contact forces remain fully simulated: the policy must physically grip and turn the nut. Thread resistance is modeled on the revolute joint by a smoothed Coulomb and viscous friction torque

$$\bar{\tau}_{f,t} = -\tau_c \tanh\left(\frac{\omega_t}{\omega_s}\right) - c_v\,\omega_t, \tag{3.5}$$

with revolute velocity ω_t, Coulomb torque τ_c, viscous coefficient c_v, and smoothing speed ω_s. Table A.1 lists the drive, friction, and pitch values. The model has one deliberate simplification beyond contact: the nut is initialized already engaged with the bolt. Thread engagement from a loose nut is a separate skill with a narrow entry configuration, and it is outside the scope of this thesis. The simplified contact opens a gap between the simulated and the physical thread, which the domain randomization of the teacher (Section 3.3) is meant to cover.

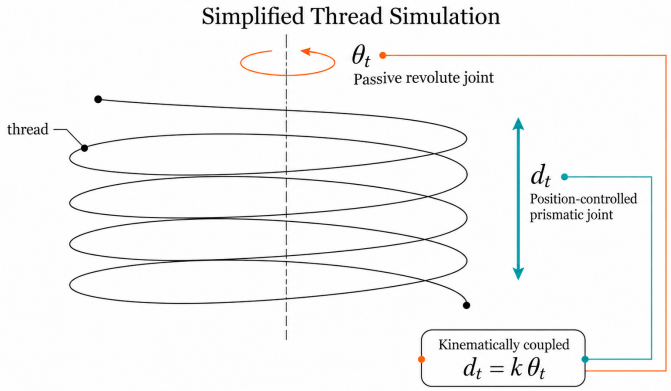


Figure 3.3: Simplified thread simulation. A passive revolute joint θ_t about the bolt axis is kinematically coupled to a position-controlled prismatic joint d_t through $d_t = k\theta_t$ with $k = -\rho/(2\pi)$, so that the nut advances by one pitch per revolution without explicit thread contacts. The finger–nut contacts remain fully simulated.

3.2.3 Action Contract and Controller

The teacher acts in a 9-D action space. A fixed action decoder, described below, maps each action to a Cartesian target for the hand's grasp frame and three pinch-joint targets. The student and the scripted transitions act in the same space and use the same decoder (Section 3.4.1). The grasp frame has its origin at the midpoint of the thumb and index fingertips. Its x axis is the thumb-to-index direction, along which the pinch closes, and its z axis is the approach direction, fixed at reset in flange coordinates. The first three components set a position offset relative to a target frame placed on the tip of the bolt. Every action component is normalized to the range from -1 to 1, and the position components are scaled by 50 mm, so the raw position action deviates by at most 50 mm per axis from this frame. Because the target is expressed relative to the bolt, the controller draws the grasped nut toward the bolt wherever nut and robot are, and a zero action does not hold the current pose. The next three components set roll, pitch, and yaw around a neutral grasp orientation, with up to ±57.3° in

roll and pitch and a yaw range of $[-180°, 90°]$, and the last three command thumb abduction, thumb flexion, and index flexion. The raw action $\boldsymbol{a}_t$ is smoothed by an exponential moving average,

$$\tilde{\boldsymbol{a}}_t = \boldsymbol{\alpha} \odot \boldsymbol{a}_t + (1 - \boldsymbol{\alpha}) \odot \tilde{\boldsymbol{a}}_{t-1}, \tag{3.6}$$

with $\alpha = 0.0625$ for the arm components and 3α for the finger components, so that a single command moves the filtered action by only a small fraction of its deviation and consecutive targets stay smooth. The filtered action is decoded to a desired pose $\boldsymbol{x}^{\mathrm{des}}$, and the target passed to the controller is clipped to a bounded step from the current pose $\boldsymbol{x}$,

$$\boldsymbol{x}^{\mathrm{tgt}} = \boldsymbol{x} + \mathrm{clip}\left(\boldsymbol{x}^{\mathrm{des}} - \boldsymbol{x}, -\boldsymbol{\delta}, \boldsymbol{\delta}\right), \tag{3.7}$$

with $\delta = 0.02\,\mathrm{m}$ per axis and $0.097\,\mathrm{rad}$ per Euler angle, applied at every physics step. One command therefore cannot demand a large jump. Table A.2 in the appendix lists the scales and limits.

The Cartesian target is tracked by the operational-space impedance controller of (2.13). The stiffness and damping gains K_p and K_d are diagonal in world axes, with $K_d = 2\sqrt{K_p}$ for critical damping, and the posture term uses the dynamically consistent nullspace projector of Section 2.1.6. The same law runs in simulation and on the physical FR3. The two implementations differ only in where the model quantities J and M come from and in how gravity is handled, which Table A.2 in the appendix compares. The finger joints are position-controlled.

3.3 Privileged Teacher Training

The privileged teacher answers difficulty (b) of Section 1.2. It learns to approach the nut, acquire the pinch grasp, keep it under load while turning, and stop at the target, all through the commanded hand and arm degrees of freedom of Section 3.2.1. Because it observes privileged simulator state, it solves this control problem without having to perceive the nut (Section 3.1).

The teacher for the learned phases in $\mathcal{M}_{\mathrm{RL}}$ is a recurrent policy, which keeps a hidden state $\boldsymbol{h}_t$ that integrates the history of past observations. It is trained with PPO (Section 2.1.2) in an asymmetric actor–critic [30], in which the critic additionally receives privileged state that exists only in the simulator and cannot be measured on the hardware. This state includes the noise-free poses of the nut and bolt, the arm joint positions, and the randomized controller gains and smoothing factor of the episode (Table 3.1). The actor receives only quantities with a counterpart on the physical system, with sensor noise applied. The critic enters only through the value estimates that shape the policy gradient and is discarded after training, so the privileged state informs the update of the actor without ever being an input to it. The actor observation $\boldsymbol{z}_t^{\pi} = g_{\pi}(\boldsymbol{x}_t, \boldsymbol{\xi}_t)$ selects the actor-visible components of the simulator state and applies observation noise $\boldsymbol{\xi}_t$. Its 46 dimensions form four groups. The pose group holds the fingertip (grasp-frame) position relative to the bolt frame and its quaternion, the end-effector linear and angular velocity, the fingertip position relative to the held nut, and the nut quaternion. The task group holds the position and quaternion error to the current turning target and a grasp-phase flag that indicates whether the proper grasp has latched. The force group holds two fingertip forces and seven arm joint torques. The last group is the previous filtered action.

The force group mirrors the sensors of the physical system. Each fingertip channel is the force that the thumb or index pad exerts along its pad normal, computed from the total contact force on the pad regardless of the body in contact, because the physical fingertip sensors observe neither the other force components nor which body presses on them. Each torque

channel is the reaction torque that the simulator measures at one arm joint, normalized by the effort limit of that joint, so that contact loads appear in it. The student observes neither channel (Section 3.5.2). Pose and force inputs are perturbed by noise that models sensor noise and calibration error (Table 3.2). The critic observes clean values of the same quantities and extends them with the arm joint positions, the complete nut and bolt pose, and the sampled controller parameters, for 76 dimensions. Table 3.1 lists the exact components.

Table 3.1: Actor and critic observations of the single-size M24 threading teacher. Noise is applied to the actor only. Joint torques are divided by the effort limits of 87 N m (joints 1–4) and 12 N m (joints 5–7).

Input	Component	Dim	Noise (actor)
actor (46-D)	fingertip position relative to bolt frame	3	$\mathcal{N}(0, 0.25\,\text{mm})$ per axis; bolt frame $\mathcal{N}(0, 1\,\text{mm})$ per reset
	fingertip quaternion (WXYZ, w and z zeroed)	4	random-axis angle $\mathcal{N}(0, 0.1°)$
	end-effector linear velocity	3	finite difference of noisy position
	end-effector angular velocity (x and y zeroed)	3	finite difference of noisy quaternion
	fingertip position relative to held nut	3	$\mathcal{N}(0, 0.25\,\text{mm})$ per axis, each body
	held-nut quaternion (WXYZ)	4	random-axis angle $\mathcal{N}(0, 0.1°)$
	target position error	3	inherits nut noise
	target quaternion error (WXYZ, $w \geq 0$)	4	inherits nut noise
	grasp-phase flag	1	none
	thumb / index flexion force	2	gain $U(0.8, 1.2)$, offset $U(-1, 1)$ N per reset; $\mathcal{N}(0, 1\,\text{N})$; ≥ 0
	arm joint torques (normalized)	7	gain $U(0.9, 1.1)$, offset $U(-0.01, 0.01)$ per reset; $\mathcal{N}(0, 0.005)$; in $[-1, 1]$
	previous filtered action (roll, pitch masked)	9	none
critic (76-D)	fingertip position, quaternion, velocities	13	clean
	arm joint positions	7	clean
	held-nut position, position relative to bolt, quaternion	10	clean
	bolt position and quaternion	7	clean
	controller proportional gains	6	sampled per reset
	action smoothing factor	1	sampled per reset
	thumb / index flexion force	2	clean
	arm joint torques (normalized)	7	clean
	position and rotation action thresholds	6	sampled per reset
	target position, quaternion, progress	8	clean
	previous filtered action	9	none

The teacher is optimized with the shaped reward

$$\begin{aligned} r_t &= \sum_{i \in \mathcal{R}} w_i^r R_{i,t} - \sum_{j \in \mathcal{P}} w_j^p P_{j,t}, \\ \mathcal{R} &= \{\text{center}, \text{face}, \text{grasp}, \text{hold}, \text{turn}, \text{target}\}, \\ \mathcal{P} &= \{\text{magnitude}, \Delta a, \text{align}, \text{force}, \text{settle}, \text{wrench}, \text{clear}, \text{contact}\}. \end{aligned} \tag{3.8}$$

The reward terms follow the three phases of the learned behavior, and Table A.3 in the appendix lists their exact form and weights. During approach, R^{center} rewards motion of the grasp center toward the nut center and R^{face} rewards motion of the thumb and index pads toward two designated opposing faces of the nut. Both pay only for newly achieved progress and stop paying once the grasp has latched, so repeated motion cannot accumulate reward. A proper grasp is simultaneous thumb and index contact near the designated faces with a pinch line nearly perpendicular to the nut axis. R^{grasp} pays a one-time bonus when the proper grasp has held for ten consecutive steps, which latches the grasp, and R^{hold} pays a small reward in every step in which the proper grasp holds under pad pressure. The hold reward of a complete episode is smaller than the target bonus, so holding without turning is worth less than threading. During turning, R^{turn} rewards newly achieved progress toward the helical target pose and pays only while the proper grasp remains live. At completion, R^{target} pays a bonus once, on the step at which the episode succeeds.

The penalties discourage large commands ($P^{\text{magnitude}}$), abrupt action changes ($P^{\Delta a}$), misalignment of the hand frame with the nut frame (P^{align}), excess grip force (P^{force}), and residual rotation near the target (P^{settle}). P^{wrench} prices the wrench that the thread joint transmits to the nut above small dead bands, except the torque about the bolt axis that threading requires. Its weight rises with the initial-yaw curriculum described below, so that the policy

learns to thread before the force limits apply. P^{clear} and P^{contact} keep the arm off the table. In addition, the effort of the three pinch joints is limited, which bounds the grip force that the policy can apply (Table A.1).

Each episode samples a target turn $\theta^* \sim U(55°, 65°)$ and builds the observed target pose at θ^*, so the policy cannot settle on one fixed stopping angle and has to read the target from its observation. An episode succeeds when the clockwise rotation lies within 2.5° of θ^* under a live proper grasp for a number of consecutive steps, so overshoot fails. Success ends the episode, so the target bonus is paid once and cannot be farmed by rocking the nut across the success angle. Once the initial-yaw curriculum is complete, the required hold grows from eight to twelve steps (0.53 s to 0.8 s). An episode also terminates when the nut leaves the bolt, tilts, or releases the thread, when the grasp is lost after it has latched, and at the 30 s time limit. The grasp-loss rule turns grasp retention into a hard constraint rather than a shaped term.

Robustness is obtained by domain randomization of the nut mass, the bolt friction, the controller parameters of Section 3.2.3, the observations, and the initial state (Table 3.2). The pose noise is small in absolute terms, because the actor's inputs are relative poses that the student will later infer from a calibrated camera. A per-episode offset of the bolt frame models calibration error and forces the teacher to close the loop on the nut rather than on the bolt frame. The initial nut rotation is randomized by an automatic curriculum that widens the yaw range from zero to ±30° in 5° steps whenever the success rate reaches 50 %, and the initial axial height of the nut is randomized as well.

Table 3.2: Domain randomization during teacher training. $U(a, b)$ denotes uniform sampling. Multiplicative factors are sampled as $1 + U(0, \ell)$ or its reciprocal with equal probability. Joint torques are in units of the joint effort limits.

Group	Quantity	Range
physics	nut mass offset	$U(-5, 5)$ g per reset
	bolt static friction	$U(0.25, 1.25)$, 128 buckets, once per environment
controller	proportional gains	multiplicative, $\ell = 0.41$, per axis and reset
	action smoothing factor	$U(0.025, 0.1)$ per reset; fingers 3×, capped at 1
	position / rotation threshold	multiplicative, $\ell = 0.25$ / 0.29
	dead zone	$U(0, [5, 5, 5, 1, 1, 1])$ N, N m, resampled every 2 s
observation	fingertip and nut position	$\mathcal{N}(0, 0.25\,\text{mm})$ per axis and step
	fingertip and nut orientation	random-axis angle $\mathcal{N}(0, 0.1°)$
	bolt frame	$\mathcal{N}(0, 1\,\text{mm})$ per axis and reset
	fingertip forces	gain $U(0.8, 1.2)$, offset $U(-1, 1)$ N per reset; $\mathcal{N}(0, 1\,\text{N})$ per step; ≥ 0
	arm joint torques	gain $U(0.9, 1.1)$, offset $U(-0.01, 0.01)$ per reset; $\mathcal{N}(0, 0.005)$ per step; in $[-1, 1]$
reward	grip-force penalty threshold	$U(5, 10)$ N per reset
initial state	nut yaw	curriculum $0 \rightarrow \pm 30°$ in 5° steps at 50 % success
	nut axial height	$U(0, 20)$ mm below nominal, room for 65° reserved
task	target turn θ^*	$U(55°, 65°)$ per episode

The demonstrations of this thesis are collected with this single-size M24 teacher. An earlier configuration of the same task, without the force group and without initial-yaw randomization, conditions one teacher on the M24, M30, and M36 nuts through a one-hot code and a round-robin assignment of sizes to environments.

3.4 Hybrid Teacher and Dataset Collection

The privileged teacher turns the nut once. The hybrid teacher answers the data side of difficulty (c) of Section 1.2: it chains the learned turn with scripted release and return, so that each phase is specified by the mechanism that specifies it most simply, and it rolls the chain out in simulation to produce complete multi-cycle demonstrations. This section

describes the cyclic rollout, the disturbance augmentation, the vision data collection, and the dataset format.

3.4.1 Cyclic Teacher Rollout in Simulation

Let $m_t \in \mathcal{M}$ denote the active phase, $\boldsymbol{x}_t$ the full simulator state, $\boldsymbol{z}_t^{\pi}$ the noisy actor observation, and $\boldsymbol{h}_t$ the actor's hidden state. During rollout the teacher is queried for its deterministic mean action $\bar{\pi}_{\mathrm{T}}(\boldsymbol{z}_t^{\pi}, \boldsymbol{h}_t) = \mathbb{E}_{\boldsymbol{a}\sim\pi_{\mathrm{T}}(\cdot|\boldsymbol{z}_t^{\pi},\boldsymbol{h}_t)}[\boldsymbol{a}]$. The hybrid teacher is

$$\pi_{\mathrm{HT}}(\boldsymbol{x}_t, m_t) = \begin{cases} \bar{\pi}_{\mathrm{T}}(\boldsymbol{z}_t^{\pi}, \boldsymbol{h}_t), & m_t \in \mathcal{M}_{\mathrm{RL}}, \\ \kappa_{m_t}(\boldsymbol{x}_t), & m_t \in \mathcal{M}_{\mathrm{script}}, \end{cases} \tag{3.9}$$

where κ_{m_t} is the scripted controller of the active transition phase. Both branches produce commands in the same filtered, bolt-relative Cartesian action space, so the recorded label is a single 9-D signal throughout the cycle. The scripted Cartesian targets are mapped back through the action decoder of Section 3.2.3, and every label is the filtered target divided by a fixed per-component scale that bounds the scripted transitions to $[-1, 1]$ (Table A.6). This unified label space is what makes the demonstration a coherent supervision signal for one student: at no step does the label change meaning or units.

Release is triggered when the clockwise nut rotation since the start of the cycle comes within 5° of the cycle's target turn, which lies between 55° and 65°. Four Cartesian waypoints follow: the first opens the thumb and index while the arm holds its pose, and the next three retract the hand away from the nut and raise it. A final return segment brings the hand back to the initial grasp pose. The return is physically gated: it counts as complete only when the grasp frame and the hand joints are back within tolerance of their reset values, and a timeout marks the cycle as a transition failure. Table A.4 in the appendix lists the thresholds and timings. While the hand is away from the nut the thread joint is held, so the nut cannot drift between cycles. On a completed return the turning reference is rebased without resetting physics. The next target turn is 60° minus the residual by which the total nut rotation misses the nearest multiple of 60°, clamped to the trained range of target turns, so that a short turn is made up in the next cycle and the nut stays on the 60° grid of the hexagon. The frozen threading policy receives the nut orientation with whole 60° steps removed, which the hexagonal nut cannot distinguish, so that its observation stays within the initial-yaw range of training. The physical nut pose is not reset, so the demonstration contains the axial advance of the nut over ten cycles. The teacher population was first qualified for one period only. Repeated cycles are a property of the hybrid rollout, not of the RL teacher alone. Figure 3.4 shows the coordinator that sequences one cycle.

Two details of the coordinator matter for the quality of the labels. First, the scripted transitions never write physics state: a transition may request that the thread joint be held while the hand is away, but it repairs nothing by teleporting the nut, so every recorded frame is a physically consistent consequence of the recorded commands. Second, the release gate is physical rather than timed: release is decided from the measured nut rotation, and the return is validated against the reset frame, so the phase labels in the recording coincide with physical events. Both properties allow the recorded phase to serve as a conditioning signal at deployment. There, the return is validated against the reset grasp pose with the same tolerances, while the release, for which the physical cell has no nut-angle sensor, is derived from the measured yaw of the grasp frame as a proxy for the nut rotation.

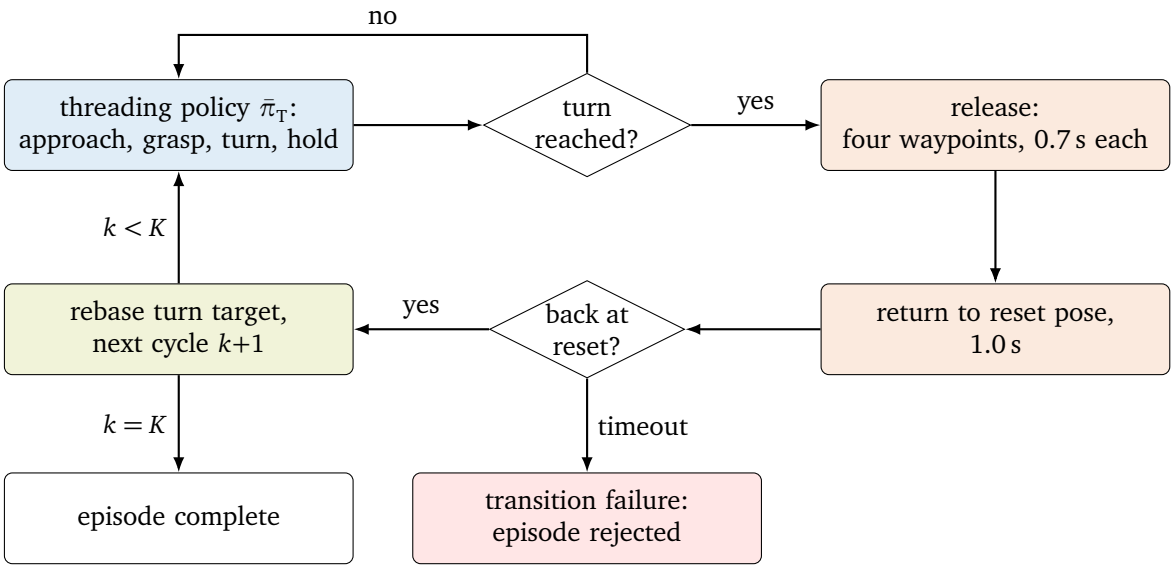


Figure 3.4: Data-collection loop of the hybrid teacher over K threading cycles. The learned policy (blue) owns approach, grasp, turn, and hold. A physical release gate on the nut rotation hands over to the scripted transitions (orange): four waypoints, which open the pinch and retreat, and the gated return. A completed return rebases the turning target and starts the next cycle, and a return that times out rejects the episode. Table A.4 lists the gate thresholds. Every policy, waypoint, and return segment receives one random arm impulse, and every executed command is recorded in the common 9-D label space.

3.4.2 Disturbance Augmentation

Twenty demonstrations contain too little variation for a student that must recover from its own deviations. Trajectory diversity is therefore increased at collection time by applying random impulses to the arm: one impulse is applied to a random arm link shortly after the start of every policy, waypoint, and return segment of every cycle, which gives 30 impulses per ten-cycle demonstration (Table A.7). The teacher, which is a closed-loop controller, reacts to the disturbance, and the recorded recovery is part of the label. The teacher is queried for its mean action and controls every step ($\beta = 1$), so each recovery segment is the teacher's own response to the state that precedes it. Besides the impulses, the demonstrations vary through the task randomization that remains active during collection (Section 3.4.4). Demonstrations in which the teacher fails to complete all cycles are rejected before materialization. The retained set consists only of explicitly successful episodes.

3.4.3 Vision Data Collection

The student perceives the scene through one RGB-D camera, so the demonstrations are rendered from a simulated copy of the physical camera. The extrinsic pose of the physical camera relative to the robot base is deduced from twelve different poses of a calibrated AprilTag observed by the camera. The resulting pose and the intrinsics are used to render the simulated camera during data collection and enter the student training pipeline. The physical stream is cropped and downscaled to the 320×180 resolution that the simulated camera renders directly. Camera geometry and field of view therefore agree between the two domains up to calibration error. Appearance and depth quality do not, because a rendered scene has one clean appearance and a rendered depth image contains no sensor noise. Texture randomization covers the first difference at collection time, and depth augmentation covers the second during training.

Texture randomization follows DextrAH-RGB [24] and varies the appearance of the rendered scene. At every reset, each environment draws new materials for the tabletop, the nut, and the bolt, each with a random texture, roughness, and metallicity. The dome light, with its texture, orientation, and intensity, and the floor texture are shared by all parallel environments and are redrawn at a reset with a fixed probability. An environment can also replace the far and invalid pixels of the RGB image with a random background image. These pixels lie outside the crop box of the point-cloud encoder (Section 3.5.2), so the background reaches only the image-space encoders of Chapter 4. The robot keeps its nominal appearance. Because the randomization is drawn at reset, the appearance is constant within a ten-cycle demonstration and differs between demonstrations. The offline loader cannot regenerate scene appearance from stored images, so this variation is recorded into the dataset once and reused unchanged by every student trained on it. Table A.7 lists the sampled ranges, and Figure 3.5 shows the resulting variation of the rendered scene.

Depth augmentation corrupts the stored metric depth images during student training. Rendered depth is exact, whereas a physical RGB-D camera returns noisy depth with missing regions, and the depth determines the position of every point that the encoder receives. The corruption is drawn anew for every training batch and acts before back-projection, so each epoch presents a different corruption of the same demonstration. It first perturbs the valid pixels with Gaussian noise whose standard deviation grows with the squared depth, adds a spatially smooth offset field, and quantizes the result. It then invalidates pixels at depth discontinuities, spatially correlated regions, and isolated pixels. Invalid pixels are rejected during back-projection, so an augmented frame yields a noisier point cloud with missing regions. The parameters in Table A.6 approximate a commodity RGB-D camera qualitatively, and Figure 3.6 shows their effect on a depth image next to a depth frame from the physical camera. The dataset of the next subsection stores the randomized RGB images together with the noise-free depth, so the same demonstrations can be trained with or without the augmentation.



Figure 3.5: Three draws of the texture randomization in the simulated workcell. The dome light and the textures of the floor and the tabletop change between draws, while the robot keeps its nominal appearance.

3.4.4 Dataset Format

The demonstrations are collected from the M24 teacher of Section 3.3 with two training mechanisms switched off: success does not end the episode, and the target turn is not sampled but follows the 60° grid of Section 3.4.1. Every environment starts from the nominal pose, without initial yaw and with the nut at the top of the thread, so that ten 60° turns, 1.67 revolutions or 5 mm of axial travel on the 3 mm pitch, fit on the bolt. All other randomization of Table 3.2 remains active, and an episode may last up to 200 s.

Each demonstration is materialized as one episode file with aligned 15 Hz samples. A sample holds the RGB and metric depth images at 320×180 pixels, the 29-D proprioception, the executed 9-D command, the active process phase, the nut pose for privileged ablations and auxiliary supervision, and the timestamp, from which the normalized task progress is

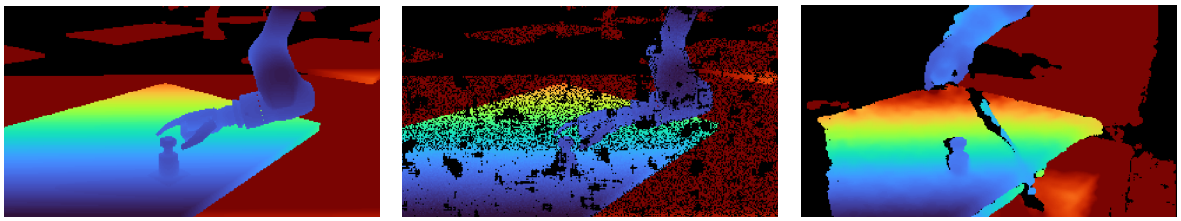


Figure 3.6: Simulated depth with and without augmentation, next to real depth. Left: the noise-free depth of one frame of a simulated demonstration, rendered by the 320 × 180 head camera. Middle: the same depth image after the full augmentation used in student training, with depth noise, quantization, and dropout of edge pixels, correlated holes, and isolated pixels. The augmentation raises the fraction of invalid pixels in this frame from 21.4 % to 47.3 %. Right: a depth frame streamed from the head camera of the physical robot, shown for comparison. All three images use the same color map, from about 0.8 m (dark blue) to 2 m (dark red), and black marks invalid pixels.

derived. Action chunks never span a reset, phase boundaries are retained so that the student learns the transitions, and terminal chunks are padded with the final command so that the last observations of an episode remain conditioning states. The retained M24 dataset contains 20 successful ten-cycle demonstrations collected from 32 parallel environments, with TBD frames in total (Table A.7).

3.5 Student Policy Distillation by Conditional Flow Matching

The student answers the deployment side of difficulty (c) of Section 1.2. The hybrid teacher cannot be deployed, because both its learned policy and its scripted controllers act on privileged simulator state (3.9). The student replaces it with one visuomotor policy that executes every phase of every cycle from sensor data, without a phase switch or separate networks. RQ1 asks whether this unified student sustains consecutive cycles better than a single-cycle policy invoked repeatedly. RQ2 asks which student design distills the demonstrations best, and Chapter 4 compares the design below with its ablations.

3.5.1 Conditional Flow Matching

The action head of the student is a conditional flow-matching model [23], [37], whose principle Section 2.1.5 introduced. This section states what the head is trained to generate and which signals it is conditioned on.

The head generates action chunks. Its training target is the demonstrated chunk of the next H commands, $\boldsymbol{U}_t^D = [\boldsymbol{u}_t, \ldots, \boldsymbol{u}_{t+H-1}] \in \mathbb{R}^{H\times 9}$. The chunk is conditioned on the student observation $\boldsymbol{y}_t = (\boldsymbol{O}_t^{\mathrm{RGBD}}, \boldsymbol{s}_t, p_t, \boldsymbol{e}_t^m)$. The RGB-D image $\boldsymbol{O}_t^{\mathrm{RGBD}}$ of the head camera shows the hand, the nut, and the bolt. The proprioception $\boldsymbol{s}_t$ holds the joint positions, the joint velocities, and the previous action. The normalized task progress $p_t \in [0,1]$ tells the student how far into the trial it is, and the process-phase encoding $\boldsymbol{e}_t^m$ marks whether the policy, the release waypoints, or the return is active. The network of Section 3.5.2 encodes these signals into one condition vector $\boldsymbol{c}_t$, which takes the place of $\boldsymbol{y}$ in (2.9). Training applies the straight-line construction of (2.8) to whole chunks:

$$\theta^* = \arg\min_\theta \mathbb{E}\left\|\boldsymbol{v}_\theta\left(\boldsymbol{U}_t^\lambda, \lambda \mid \boldsymbol{c}_t\right) - \dot{\boldsymbol{U}}_t^\lambda\right\|_2^2, \qquad \boldsymbol{U}_t^\lambda = (1-\lambda)\boldsymbol{U}_t^0 + \lambda \boldsymbol{U}_t^D, \quad \dot{\boldsymbol{U}}_t^\lambda = \boldsymbol{U}_t^D - \boldsymbol{U}_t^0, \quad (3.10)$$

where the expectation is over $(\boldsymbol{y}_t, \boldsymbol{U}_t^D) \sim \mathcal{D}$, a Gaussian chunk $\boldsymbol{U}_t^0 \sim \mathcal{N}(\boldsymbol{0}, I)$ of the same shape, and $\lambda \sim U[0,1]$. At run time, the student draws a Gaussian chunk $\boldsymbol{U}^0$ and follows the learned

field with N explicit Euler steps, $\boldsymbol{U}^{\lambda+1/N} = \boldsymbol{U}^{\lambda} + \frac{1}{N}\boldsymbol{v}_\theta(\boldsymbol{U}^{\lambda}, \lambda \mid \boldsymbol{c}_t)$, with $N = 16$ in simulation (Table A.6 lists the deployment setting). The condition $\boldsymbol{c}_t$ is computed once per chunk, so the image is encoded once and each Euler step evaluates only the Transformer of Section 3.5.2.

A generative head is used because the demonstrations are multimodal. The teacher's recoveries after impulses depend on state that the observation does not fully resolve, so near-identical observations can carry the nominal action in one demonstration and a recovery in another. A regression head averages such labels, whereas the flow head samples one of them (Section 2.1.5). Among generative heads, flow matching is chosen over diffusion because it trains on a plain regression target without a noise schedule and samples in a few deterministic integration steps.

3.5.2 Student Architecture

Figure 3.7 shows the network architecture of the student, which encodes the observation $\boldsymbol{y}_t$ into the condition $\boldsymbol{c}_t$ and predicts the flow velocity $\boldsymbol{v}_\theta$ of the action chunk from it. It consists of a vision encoder, a proprioception encoder, a fusion and conditioning stage, and the velocity network.

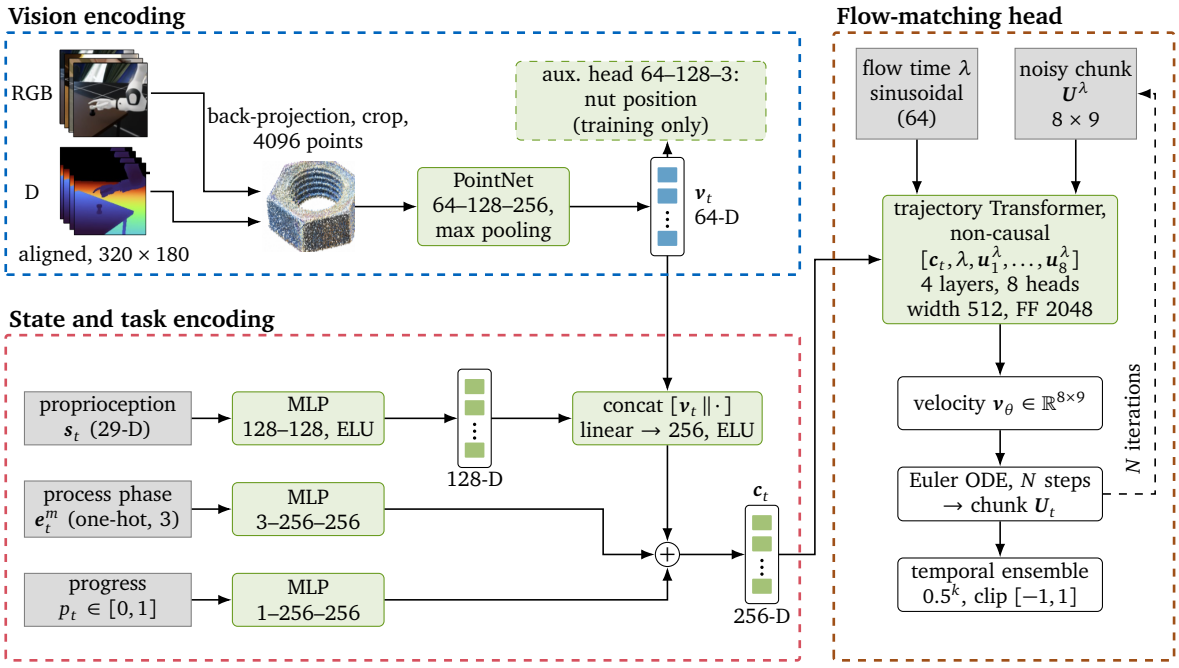


Figure 3.7: Student architecture. The aligned RGB-D observation is back-projected into a point cloud and encoded into a 64-D visual feature. The 29-D proprioception is encoded by an MLP. Late fusion concatenates the two with the visual feature first and projects to the 256-D condition, to which learned embeddings of the global trajectory progress and of the process phase are added. A non-causal trajectory Transformer predicts the flow velocity of all eight action tokens given the condition token and a sinusoidal embedding of the flow time. An auxiliary head regresses the nut position from the visual feature during training.

The student encodes the visual and proprioceptive streams separately and fuses them by concatenation, following DextrAH-RGB [24], with the visual feature placed before the encoded proprioception. The 29-D proprioception $\boldsymbol{s}_t$, consisting of the ten controlled joint positions, the ten joint velocities, and the previous filtered 9-D action, is encoded by an MLP, and the concatenation of the 64-D visual feature and the proprioceptive feature is projected to the 256-D condition.

The vision encoder of the student is the point-cloud encoder of DP3 [26]. The depth

image is back-projected through the calibrated intrinsics into camera-frame points. Points outside a fixed crop box around the workspace or outside the valid depth range are rejected, and the remaining points are normalized by fixed workspace constants and carry the color of their pixel. The encoder takes a fixed set of 4096 points per frame, while the number of valid points varies from frame to frame. Each pixel therefore receives a ranking score, and the 4096 highest-ranked pixels form the input set. Valid pixels always rank above rejected ones, so rejected pixels enter the set only when a frame has fewer than 4096 valid points. These fill-in points stay marked as invalid and are excluded from the pooling below. Among the valid pixels, the score decides which subset is kept. During training it is drawn uniformly at random, so every update sees a different subset of the scene. At evaluation each pixel carries a fixed score from a golden-ratio sequence over the pixel index, which makes the selection deterministic and spreads it across the image without the cost of farthest-point sampling. A compact PointNet with pointwise layers and global max pooling over the valid points yields the 64-D visual feature $\boldsymbol{v}_t$. The encoder has no generic pretrained backbone. It is pretrained on the task scene and then fine-tuned inside the student, as described below. The point-cloud encoder is first trained alone, on a pose-regression task in parallel simulated scenes, and then fine-tuned end to end inside the student. The pretraining scenes contain the workcell table, the nut and bolt, and the calibrated camera, but no robot. In every scene a free-floating nut follows a random six-DoF path through the visible workspace, and the rendered RGB-D frames are consumed immediately by the encoder and a pose decoder. The decoder predicts the camera-frame nut position and orientation with a loss that treats the six 60° rotations of the hexagonal nut as equivalent. Only the encoder weights are transferred to the student, and the pose decoder is discarded. All encoder parameters remain trainable and are optimized jointly with the flow head, so the representation adapts to the hybrid-teacher scenes and to the action objective. Table A.6 lists the pretraining budget and its validation errors.

Two low-dimensional conditions, the global trajectory progress and the process phase, are added to the 256-D condition through separate learned embeddings. The global trajectory progress $p_t \in [0, 1]$ maps the first timestamp of each demonstration to 0 and the last to 1. At deployment it advances with elapsed time from the start of the trial, normalized by the median demonstration duration stored in the checkpoint, and is clamped to 1 afterwards. It is a monotonic clock for the complete behavior and nothing more: it is not a teacher-stage selector, it is not periodic, and it does not encode the cycle index. It tells the student how far into a nominally timed trial it is, which disambiguates, for instance, the first and the last approach of a trial that otherwise look alike. The process-phase encoding $\boldsymbol{e}_t^m$ is a three-way one-hot over the policy-controlled phases, the release waypoints, and the return. During training it is derived from the recorded phase labels, and during evaluation it is published by the cyclic coordinator, in simulation from the physical events of Section 3.4.1 and on the robot from the release proxy and return gate, so that a cycle that takes longer than nominal cannot select the wrong phase. The phase encoding is a synchronization aid: it does not switch networks or action contracts, and the student predicts the transition actions itself.

The velocity field $\boldsymbol{v}_\theta$ is parameterized by a non-causal trajectory Transformer. The noisy chunk $\boldsymbol{U}^\lambda$ is split into $H = 8$ action tokens of dimension 9. A condition token embeds $\boldsymbol{c}_t$, and a time token embeds the flow time λ through a sinusoidal encoding. Learned positional embeddings mark the token slots and carry no absolute time. The Transformer applies full self-attention over all tokens, so that every step may attend to the whole noisy chunk when predicting its velocity, and a linear output layer maps each action token to its 9-D velocity. The model keeps an exponential moving average of its parameters, and inference uses the averaged weights. An auxiliary head on the visual feature regresses the nut position. It regularizes the visual feature toward spatially meaningful content and is discarded at deployment. Table A.6 in the appendix lists the network sizes and the training hyperparameters.

The student predicts actions in chunks of $H = 8$ steps, which cover 0.53 s at the 15 Hz control rate. The first action of a chunk is the command that acts on the observed state. At deployment the student replans at every control step, and the command sent to the controller is a temporal ensemble of all still-valid proposals for the current tick with weights 0.5^k for a proposal of age k. A decay of zero recovers strict first-action execution. The ensemble operates in action space and is distinct from the parameter-space EMA. The executed 9-D action is clipped to $[-1, 1]$, decoded to the filtered Cartesian target, and passed to the controller of Section 3.2.3. Because the label scale is fixed, no action statistics are learned.

3.5.3 Offline Training

Algorithm 1 summarizes offline distillation. The dataset consists of the simulated hybrid-teacher rollouts, windows of H consecutive commands are built from each episode without crossing episode boundaries, and the velocity field is trained with (3.10).

Algorithm 1 Offline flow distillation

Require: hybrid teacher π_{HT}, horizon H

1: Collect simulated rollouts $\mathcal{D}_{\mathrm{sim}}$ with π_{HT} under random arm impulses and visual domain randomization
2: Form H-step chunks from $\mathcal{D} = \mathcal{D}_{\mathrm{sim}}$, masking resets and padding terminal chunks
3: **repeat**
4: Sample $(\boldsymbol{y}_t, \boldsymbol{U}_t^D) \sim \mathcal{D}$, $\boldsymbol{U}_t^0 \sim \mathcal{N}(\mathbf{0}, I)$, $\lambda \sim U[0, 1]$
5: Encode the point cloud and fuse it with proprioception, progress, and phase to obtain $\boldsymbol{c}_t$
6: Update θ by one gradient step on (3.10) plus the auxiliary nut-position loss
7: Update EMA parameters $\bar{\theta} \leftarrow \alpha\bar{\theta} + (1-\alpha)\theta$
8: **until** convergence
9: **return** student velocity field $\boldsymbol{v}_{\bar{\theta}}$

With every component of the method defined, Chapter 4 evaluates it in simulation to answer RQ1 and RQ2, and Chapter 5 evaluates it on the physical system to answer RQ3.

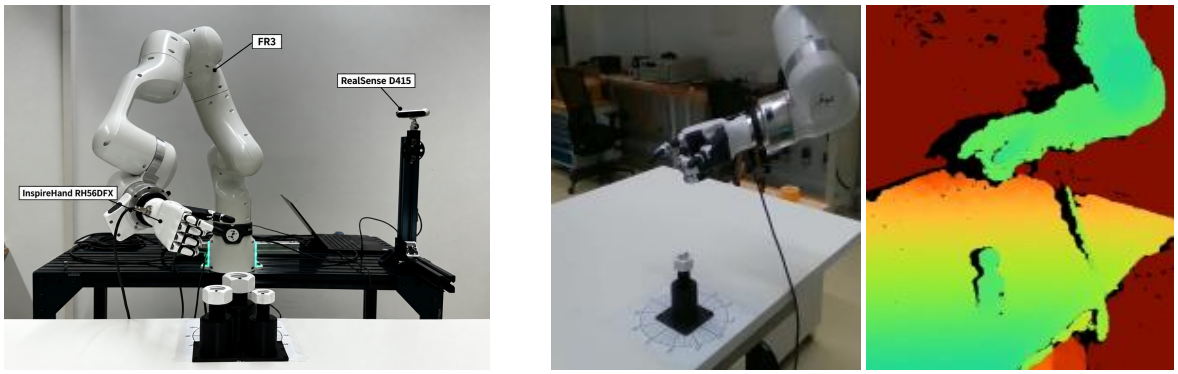
Chapter 4

Experiments in Simulation

This chapter tests the method of Chapter 3 in simulation, where every component can be varied under matched conditions. It answers RQ1 and RQ2 of Section 1.4. RQ3, the zero-shot transfer, needs the physical system and is evaluated in Chapter 5. Section 4.1 lists the hardware, and Section 4.2 defines the metrics with which this chapter and Chapter 5 report their results. Section 4.3 reports the ablations that answer RQ1 and RQ2, and Section 4.4 summarizes the chapter. Numbers that are not yet available are marked TBD.

4.1 Experimental Setup

Teacher and student policies are each trained on a single NVIDIA GeForce RTX 4090 GPU. At inference time, the student runs with CUDA on an NVIDIA GeForce RTX 3060 Mobile GPU. The physical platform consists of the FR3 arm, the Inspire RH56DFX hand, and an Intel RealSense D415 camera (Section 3.2.1). Figure 4.1 shows the workcell and the camera observation.



(a) Physical workcell.

(b) Live RGB (left) and depth (right) observation.

Figure 4.1: Hardware setup. Left: the FR3 arm with the Inspire RH56DFX hand, the bolts on their holders in front of the robot, and the fixed RealSense D415 on its stand. Right: the RGB image and the metric depth image of the D415 as the student receives them, after center-cropping and downscaling to 320×180.

4.2 Evaluation Protocol

Unless a study states otherwise, each condition is evaluated over $N_{\mathrm{tr}} = 10$ trials with up to ten cycles per trial, where each cycle consists of a nominal 60° clockwise turn followed by

release and return. A cycle counts as completed when the release has been triggered and the return has been validated against the reset pose within its 16 s timeout (Section 3.4.1). A trial ends at the first transition failure, early termination, or time limit. Three quantities are reported. The mean completed cycle count $\bar{C} = N_{\text{tr}}^{-1} \sum_i C_i$ counts, for trial i, the cycles $C_i \in \{0, \ldots, 10\}$ completed before failure or timeout. Incomplete cycles are excluded, and higher is better. The cyclic tracking error is computed for every completed cycle k of trial i as $e_{i,k} = \Delta\varphi_{i,k} - 60°$, where $\Delta\varphi_{i,k} = \varphi_{i,k} - \varphi_{i,k-1}$ is the clockwise nut rotation. Over all $M = \sum_i C_i$ completed cycles the mean $\mu_{\text{cyc}} = M^{-1} \sum_{i,k} e_{i,k}$ and standard deviation σ_{cyc} are reported, where negative values indicate lag and positive values overshoot, and a mean closer to zero with a smaller deviation is better. If $M = 0$ both statistics are undefined. The ten-cycle tracking error is computed over the set $I_{10} = \{i : C_i = 10\}$ of trials that complete all cycles as $e_{10,i} = \sum_{k=1}^{10} \Delta\varphi_{i,k} - 600°$, reported as mean μ_{10} and standard deviation over I_{10}. All errors are in degrees. In simulation the nut angle is read from the simulator. On hardware it is read from an angular scale mounted under the bolt (Chapter 5).

4.3 Student Ablations in Simulation

The main ablation study answers RQ1 and RQ2 of Section 1.4. Five students are trained on the simulated dataset of Section 3.4.4 and evaluated under matched budgets, and each ablation replaces one part of the full method: the training data in (a), the encoder in (b), and the generative action head in (c) and (d), where (c) also replaces the Transformer backbone. Ablation (a) addresses RQ1: it tests whether one student trained on multi-cycle data outperforms a single-cycle policy that is invoked repeatedly. Ablations (b) to (d) address RQ2: (b) replaces the point-cloud encoder with an image-space encoder, and (c) and (d) replace the flow-matching head with deterministic regression on a recurrent and on a Transformer backbone. RQ3 is evaluated on hardware in Chapter 5.

(a) *Single-cycle student.* The student is trained on demonstrations of one cycle and is invoked ten times in a trial. Its temporal context is reset after every cycle.

(b) *ResNet visual encoder.* The point-cloud encoder is replaced by a one-branch image-space encoder. An ImageNet-pretrained ResNet-18 stage-2 trunk processes RGB, log depth, and a validity mask. An embedding of the calibrated camera-frame XYZ is added on the token grid, and attention pooling over the tokens yields the same 64-D feature. Policy, objective, data, and budget are unchanged.

(c) *Point-cloud encoder with LSTM regression.* The flow-matching Transformer is replaced by the DextrAH-RGB-style recurrent backbone, a layer-normalized 512-unit LSTM followed by a 512–512–256 ELU MLP, trained by deterministic action regression on the same data.

(d) *Point-cloud encoder with Transformer regression.* A deterministic variant of ACT [33] without the variational objective predicts the eight-step chunk from eight learned action queries. Its backbone has the same size as the flow Transformer.

(e) *Full method.* One flow-matching student with the point-cloud encoder is trained on the complete multi-cycle simulated demonstrations.

Table 4.1 reports the results on the M24 hexagonal nut. The full method (e) completes all ten cycles in every trial, with a cyclic tracking error of $-1.77 \pm 1.02°$ and a ten-cycle error of $-17.7 \pm 3.23°$. The student thus lags the 60° target by less than two degrees per cycle, and the

lag accumulates over the trial without its spread drifting. Each ablation removes one property on which this result may depend. The single-cycle student (a) never sees the axial advance of the nut: in a single-cycle demonstration the nut is always grasped at its reset height, whereas over ten cycles it advances down the bolt by one sixth of a pitch per cycle, and the multi-cycle student has to read this height from the point cloud (RQ1). The ResNet student (b) keeps everything but the representation. The regression students (c) and (d) keep the data and, for (d), also the backbone, the chunk length, and the temporal ensembling, so that their difference to (e) isolates the generative objective (RQ2). Their results on the simulated dataset are TBD.

Table 4.1: Simulation ablations on the M24 hexagonal nut over ten trials. Each defined entry reports mean ± standard deviation. Completed cycles are averaged across trials, cyclic-error statistics use completed cycles only, and the ten-cycle error compares the accumulated rotation with the 600° target. It is reported as "–" when no trial completes all ten cycles. Errors are signed in degrees: negative denotes lag and positive overshoot.

Ablation	Completed cycles ↑	Cyclic error → 0	Ten-cycle error → 0
(a) single-cycle student	TBD	TBD	TBD
(b) ResNet encoder	TBD	TBD	TBD
(c) LSTM regression	TBD	TBD	TBD
(d) Transformer regression	TBD	TBD	TBD
(e) full method	10.0 ± 0.0	-1.77 ± 1.02	-17.7 ± 3.23

For the encoder part of RQ2, a representation probe complements the closed-loop ablation (b). One shared pose decoder is trained on the frozen 64-D feature of each encoder to predict the nut position and orientation. A re-grasp needs exactly the orientation of the nut faces relative to the fingers, so the probe tests whether a representation carries the quantity on which the closed loop depends. The probe errors of the point-cloud and the ResNet features are TBD.

4.4 Summary

In simulation, the full method trained on the demonstrations of the hybrid teacher completes all ten cycles of the M24 task in every trial with a cyclic error of $-1.77 \pm 1.02°$. Whether the multi-cycle data (RQ1) and the point-cloud encoder and flow-matching head (RQ2) are each necessary will be settled by the reruns of ablations (a) to (d) on the simulated dataset. Simulation cannot show whether the student bridges the gap in contact and perception that the simplified thread model and the rendered images leave open. Chapter 5 therefore deploys the same student, trained on simulated data only, on the physical system to answer RQ3.

Chapter 5

Experiments on Real Hardware

Chapter 4 showed that the distilled student sustains consecutive threading cycles in simulation. This chapter asks whether that behavior survives the gap between the simplified simulation and the physical workcell, which is RQ3 of Section 1.4: can a student trained on simulated demonstrations only transfer zero-shot to the physical system? No real-world data enters training, and nothing is trained on the robot. Section 5.1 introduces the physical system, including the robot, the camera and its calibration, the controller, the measurement of the nut angle, and the test objects. Section 5.2 describes the zero-shot transfer protocol. Section 5.3 reports the closed-loop results on the M24 training nut, Section 5.4 on unseen nut geometries, and Section 5.5 under long-horizon stress.

5.1 Hardware Setup

The physical workcell, shown in Figure 4.1, is the platform of Section 3.2.1: the FR3 arm with the Inspire RH56DFX right hand and the three commanded pinch degrees of freedom described there. The bolt stands on a 5 cm pedestal on a 60 cm by 80 cm bench, and the robot base is mounted at tabletop height, 46.5 cm from the near table edge, so that the reach envelope lies above the work surface.

The simulated system is a model of this workcell. The transform of the adapter flange was measured and reproduced in the simulated asset. The hand asset was regenerated from the official robot URDF that ships with this hand, which carries the measured pivot geometry of the fingers. Joint limits and link inertias are those of the URDF, and linear mimic ratios approximate the linkage that couples the finger joints of the physical hand. The FR3 is the arm that replaced the earlier humanoid-arm platform of Section 3.2.1.

A fixed Intel RealSense D415 on a stand provides the RGB-D observation at 640×480. As in simulation, the stream is center-cropped and downscaled to the 320×180 policy input (Figure 4.1). The pose of the camera relative to the robot base was deduced from twelve different poses of a calibrated AprilTag observed by the camera, with a translation fit RMSE of 27 mm and a rotation fit RMSE of 3.5° over the calibration poses. The calibrated optical pose and the intrinsics are the ones used to render the simulated camera and to back-project the point cloud, so the student sees the same geometry in both domains.

Robot commands use the same Cartesian impedance law as in simulation, so the student's bounded action reaches the arm through the shared action path of Section 3.2.3. The deployment controller is derived from the libfranka Cartesian-impedance example controller and modified to follow (2.13), including the axis-angle orientation error and the dynamically consistent nullspace term. The modifications are described below. Gravity is compensated by libfranka, which matches the simulation, where gravity is disabled for the arm. The con-

troller runs at the 1 kHz rate of the Franka control interface, while the student commands targets at 15 Hz. The hand is position-controlled through its serial interface. The task-space gains equal those of the simulation, 565 N/m and 28 N m/rad with critical damping, and the nullspace stiffness is 10. The mapping of the student's bounded action to the controller target is TBD.

The unmodified libfranka example differs from the Factory controller of (2.13) in three places. It uses a damped kinematic nullspace projector built from a pseudoinverse of the Jacobian transpose with regularization 0.2, it feeds the rotational spring the vector part of the error quaternion instead of the rotation vector, and it adds the robot's Coriolis term. Near the threading posture, the damped projector lets the posture spring push 12.5 N on the tool at a 1 rad wrist excursion, comparable to the 11.3 N that the policy commands for a 20 mm position step, whereas the exact projector reduces this to 0.4 N. The quaternion vector halves the effective rotational stiffness from 28 to 14 N m/rad. The deployment controller therefore replaces all three: it uses the dynamically consistent projector with the arm mass matrix from the robot model, with the hand configured as end-effector load, the rotation-vector error, and no Coriolis term, because the simulated arm is not compensated either. Both backends are implemented in the simulation with selectable gains, so a teacher can be trained under either. The reported teachers use the Factory controller, and the physical controller reproduces it rather than the other way around. The student's commands therefore meet the stiffness under which the teacher acquired its contact behavior.

For open-loop validation of the action path, every simulated demonstration can be exported as a command-only deployment artifact. The simulator recording is authoritative for the geometric joint path. Some scripted phases, however, change the joint position kinematically while writing zero articulation velocity, so the export does not use recorded velocities. It fits one twice continuously differentiable quintic curve through the recorded positions and derives position, velocity, and acceleration from it, with deployment limits on joint positions and velocities applied explicitly. On the FR3 the Cartesian targets are executed directly through the libfranka-style controller, which keeps open-loop playback and student execution on the same path.

The metrics of Section 4.2 require the nut angle. It is read from an angular scale printed on a sheet under the bolt pedestal (Figure 5.1), and the unwrapped angle is recorded at the end of every goal-reaching phase. The test-object family, shown on the right of Figure 4.1, consists of M24, M30, and M36 nuts in four profiles each, hexagonal, octagonal, rectangular, and round, with matching bolts and holders. Teacher and student training use the hexagonal profiles only. The other three profiles are held out for the generalization study of Section 5.4.

5.2 Zero-Shot Transfer Protocol

The protocol tests RQ3 directly: the student is trained on simulated data only and deployed unchanged. The student of the full method is trained for 300 epochs on the simulated dataset $\mathcal{D} = \mathcal{D}_{\text{sim}}$ of Section 3.4.4 with eight-step chunks. It is deployed on the physical robot with a prediction horizon of eight and sixteen ODE integration steps. No real-world images, trajectories, or corrections enter its training, and it is not fine-tuned on the robot. The transfer therefore rests entirely on four properties of the pipeline. The calibrated camera of Section 5.1 is the one that rendered the demonstrations. The point-cloud crop box removes the physical background before encoding. The demonstrations were rendered under lighting, texture, and background randomization. The Cartesian targets are executed by the impedance controller described above. The student receives the progress scalar from the elapsed trial time and the process phase from the physical events of the cycle, as in simulation (Section 3.5.2).

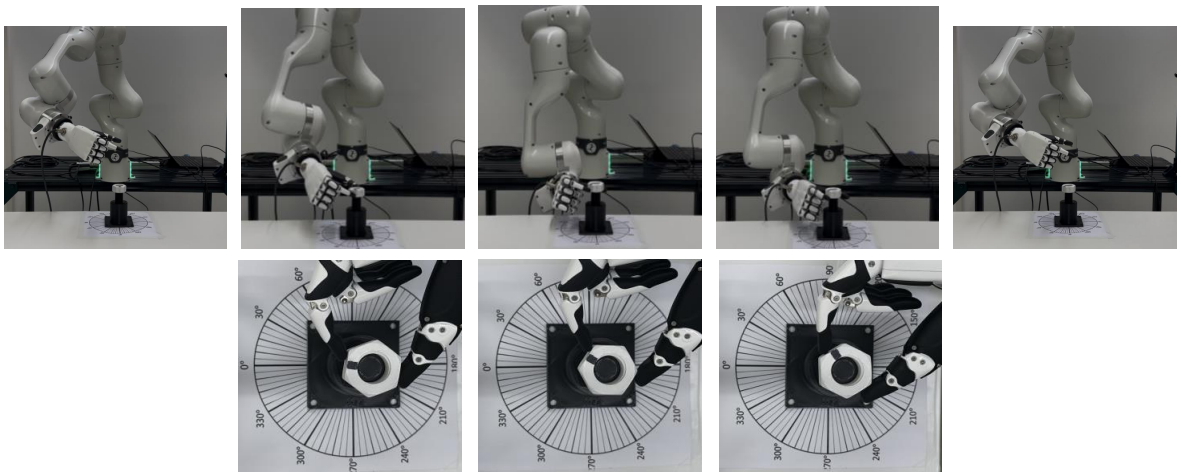


Figure 5.1: One threading cycle on the physical robot. Top, from left to right: approach, grasp, rotation, goal reaching, and release with return to the reset pose. Bottom: top-down views of the angular scale under the bolt during grasp, rotation, and goal reaching. The scale is read at the end of every goal-reaching phase to compute the cyclic and ten-cycle tracking errors.

5.3 Results on the Training Nut

Table 5.1 is the hardware counterpart of the simulation ablation on the M24 hexagonal nut. It answers RQ3 on the training nut, and it checks whether the comparisons of RQ1 and RQ2 carry over to hardware. Three of the five students are taken to hardware: the full method (e), the single-cycle student (a), and the ResNet student (b). The single-cycle student tests whether the advantage of multi-cycle distillation persists on the physical system, and the ResNet student tests whether the image-space representation transfers as well as the point cloud. The two regression heads (c) and (d) are evaluated in simulation only. The hardware results are TBD.

Table 5.1: Real-world zero-shot ablations on the M24 hexagonal nut over ten trials, with the same conventions as Table 4.1. Ablations (c) and (d) are evaluated in simulation only.

Ablation	Completed cycles ↑	Cyclic error → 0	Ten-cycle error → 0
(a) single-cycle student	TBD	TBD	TBD
(b) ResNet encoder	TBD	TBD	TBD
(e) full method	TBD	TBD	TBD

5.4 Generalization across Nut Sizes and Geometries

The training nut alone does not show how far the zero-shot transfer reaches, so the full method is also evaluated on all twelve test objects. Table 5.2 reports ten trials per object: the M24, M30, and M36 hexagonal nuts, whose profile the demonstrations show, and the octagonal, rectangular, and round profiles of each size, which it has never seen. The study tests two kinds of generalization. Across sizes, the student must infer the nut size from the point cloud, since it receives no explicit size input. Across profiles, the question is whether the learned pinch-and-re-grasp strategy, which the demonstrations show only on hexagons, transfers to a different face topology. The results are TBD.

Table 5.2: Real-world zero-shot transfer across nut sizes and profiles over ten trials per nut. Values are mean ± standard deviation. The octagonal, rectangular, and round profiles and the M30 and M36 sizes are absent from the demonstrations.

Size	Profile	Completed cycles ↑	Cyclic error → 0	Ten-cycle error → 0
M24	hexagonal	TBD	TBD	TBD
	octagonal	TBD	TBD	TBD
	rectangular	TBD	TBD	TBD
	round	TBD	TBD	TBD
M30	hexagonal	TBD	TBD	TBD
	octagonal	TBD	TBD	TBD
	rectangular	TBD	TBD	TBD
	round	TBD	TBD	TBD
M36	hexagonal	TBD	TBD	TBD
	octagonal	TBD	TBD	TBD
	rectangular	TBD	TBD	TBD
	round	TBD	TBD	TBD

5.5 Long-Horizon Stress

Beyond the ten-cycle protocol, the student is run on hardware for as many cycles as it sustains, to measure how far its horizon extends beyond the ten cycles of the demonstrations. Every revolution advances the nut by one pitch down the bolt. After enough cycles, the nut therefore leaves the axial range that the ten-cycle demonstrations cover. The stress trials thus measure how far the learned behavior reaches along the bolt. The cycle counts per nut are TBD.

Together, the three studies provide the hardware evidence for RQ3: transfer on the training nut, generalization across sizes and profiles, and the reach of the behavior beyond the demonstrated horizon. Chapter 6 interprets this evidence alongside the simulation results of Chapter 4.

Chapter 6

Discussion

This chapter interprets the results of Chapters 4 and 5. Section 6.1 answers the three research questions of Section 1.4 as far as the available results allow. Section 6.2 draws the lesson of the action and observation contracts. Section 6.3 returns to the assembly motivation and asks how far the demonstrated skill is from industrial threading. Section 6.4 states the limitations of the work.

6.1 Interpretation per Research Question

All three research questions presuppose a teacher that produces complete multi-cycle demonstrations, so the teacher result comes first. PPO learns the threading turn once the thread is constrained kinematically and the learned task is one turn from an engaged nut. The kinematic constraint makes the learning problem well posed: it ties axial progress to rotation, so the progress reward can only be earned by turning. The teacher also answers difficulty (b) of Section 1.2.2: with the hand restricted to three commanded pinch degrees of freedom, it acquires the pinch grasp and keeps it under the turning load, and losing the grasp after it has latched ends the episode, which makes grasp retention a hard constraint (Section 3.3). Trained under initial-yaw, axial-height, and controller randomization, the retained teacher, chained into the hybrid teacher, produces the successful ten-cycle demonstrations of the student dataset (Section 3.4.4). The RL teacher solves one turn, and the hybrid rollout extended it to ten consecutive cycles. The turn also needs no force sensing: the actor of the retained teacher observes no force channel. Hardware-equivalent force observations are implemented for teacher and student and remain to be evaluated.

RQ1 asks whether one student distilled from multi-cycle demonstrations sustains consecutive cycles, and it tests the deployment side of difficulty (c) of Section 1.2.3. The full method completes all ten cycles in every simulated trial and lags the target by less than two degrees per cycle. The content of the data is the likely reason. In a single-cycle demonstration the nut is always grasped at its reset height, so a student trained on it never has to read the height from the point cloud. In a multi-cycle demonstration the nut descends by one sixth of a pitch per cycle, and the student learns to track it. The global progress scalar supports the multi-cycle student, but it does not supply the axial information. That information comes from the coverage of the axial state in the demonstrations. The single-cycle ablation (a) receives the same progress scalar and therefore isolates this effect. Its result is TBD. RQ1 thus tests the principle that unifying the complete cyclic behavior in one dataset, rather than invoking a per-cycle policy repeatedly, yields a policy that sustains consecutive cycles.

RQ2 asks which student design makes the unification work. It has two parts, the visual representation and the action head, and each of the ablations (b) to (d) changes one of them.

On the representation side, the student does not need to localize the nut to sub-millimeter accuracy. The engagement of the nut on the bolt and the compliance of the impedance controller absorb lateral error. What the student needs is the orientation of the nut faces for the re-grasp and the height of the nut for the approach. A representation that encodes position but not orientation would let the fingers close on the right spot with the wrong face alignment. The point-cloud encoder consumes the metric geometry of nut, bolt, and hand directly. It has about 60 000 parameters and no pretrained image backbone. The pose probe of Section 4.3 tests whether the feature of each encoder carries the face orientation. The image-space encoder of ablation (b) also receives depth and per-pixel coordinates, so (b) tests whether the geometric structure of the representation, rather than the depth input alone, decides the closed-loop result. Random point sampling adds an augmentation that the image encoder does not receive. The closed-loop ablation (b) and the pose probe quantify the difference. Their results are TBD.

On the head side, the chunked Transformer regression head (d) shares the backbone, the chunk length, and the temporal ensembling of the flow student, so the difference between (d) and (e) isolates the generative objective. Two mechanisms could favor the flow head. The first is multimodality of the labels. For near-identical observations, the dataset contains the teacher's nominal action in one demonstration and its recovery after an impulse in another. A squared-error head converges to their average, an action that neither demonstration contains, whereas the flow head samples one of them. The second is the structure of the prediction. The flow head predicts a velocity from a noisy chunk toward the data, a denoising problem that may be easier to fit than a direct regression from observation to chunk. Through the sixteen integration steps it refines all eight steps of the chunk jointly, with full attention among them. The recurrent regression head (c) additionally removes chunking. The results of (c) and (d) on the simulated dataset are TBD.

RQ3 asks whether a student trained on simulated data only transfers zero-shot to the physical system, and it tests the transfer side of difficulty (a) of Section 1.2.1. The transfer rests on four properties of the pipeline. The calibrated camera model is meant to make the simulated and real point clouds geometrically comparable. The crop box removes the background, which is where rendered and real images differ most. Lighting, texture, and background randomization, together with image-space color augmentation, are meant to cover the remaining appearance variation of the points that enter the encoder. The shared Cartesian action path executes the student's commands through a controller that reproduces the simulated control law, and the teacher was trained under randomized gains and noisy pose inputs. The same geometric property that RQ2 relies on matters here: inside the crop box the point geometry is identical in both domains up to depth noise and calibration error, whereas an image-space encoder faces the appearance gap of the full image. The per-point colors still carry the appearance gap of nut, bolt, and hand. The approach requires no real-world data collection and no training on the robot. The hardware results that test it are TBD.

6.2 Action and Observation Contracts

The student imitates the teacher's bounded, bolt-relative Cartesian action. This keeps it in the teacher's own action space, expresses every target relative to the bolt tip, and executes the command through the unchanged controller, including the action smoothing that the teacher was trained with. A student should therefore imitate the action the teacher produced, in the units the teacher produced it, and let the controller do the conversion it was designed for.

The observation contract follows the same logic. The student receives the previous filtered

action, which the teacher also observes, the positions and velocities of the ten controlled joints, which the teacher actor does not observe, and two low-dimensional conditions that the demonstration provides without privileged state. The global progress scalar assumes a nominally timed trial. For trials that run much longer than the demonstrations, the phase should be inferred from the observation history inside the policy rather than supplied as an external signal. Chapter 7 returns to this as future work.

6.3 What This Means for Robotic Assembly

The motivation of this thesis was assembly with threaded joints (Section 1.1). Measured against an industrial fastening step, the demonstrated skill covers the run-down of an engaged nut and leaves four gaps.

The first gap is the start of the task. Every trial starts with the nut already engaged on the bolt (Section 3.2.2). In assembly the nut arrives loose and has to be seated on the thread first, which is a separate skill with a narrow entry configuration. Engagement is therefore not covered by the demonstrated skill.

The second gap is the end of the task. A trial ends after ten turns of 60°, which advance the nut by one and two thirds of a pitch, 5.0 mm on the M24 nut. The nut is never seated against a part, and it is never tightened to a preload. The simulated thread resists the turn only with a smoothed friction torque whose Coulomb term is 2×10^{-3} N m, which models a free-running nut. The actor observes no force, and the reward penalizes excess wrist force. A tightening phase would demand that the pinch transmit a much larger torque and that the policy sense when the joint is tight, which neither the thread model nor the observation contract supports.

The third gap is the grasp. The student executes one strategy, a thumb–index pinch with three commanded hand joints, learned on the face topology of a hexagon. People change the grasp as the resistance grows (Section 1.2.2), and a tightening phase would likely require such a change. The held-out octagonal, rectangular, and round profiles of Section 5.4 test how far the single strategy carries. Their results are TBD.

The fourth gap is cycle time. In the demonstrations, release and return take a nearly fixed time in every cycle, since their scripted waypoints are interpolated over 0.7 s per segment and 1.0 s for the final return. Because each cycle turns the nut by 60°, a full revolution takes six cycles, which is far slower than the continuous rotation of a powered spindle.

These gaps do not erase what the result shows. In simulation, a learned policy controlled a dexterous hand through ten cycles of grasp, turn, release, and return, and it re-grasped the nut at the angle and height that the previous cycle left it. It perceives the nut from one fixed camera, and it is trained without any real data. The same hand also picks up a loose nut in simulation. Whether this versatility outweighs the speed of a dedicated tool depends on the application, and the remaining gaps define the work that separates the two (Chapter 7).

6.4 Limitations

Section 6.3 described the task-level gaps to industrial threading. This section states the limitations of the method and of the evidence. The demonstrated behavior is a pinch-grasp re-grasp strategy, and the student has no mechanism to discover a different grasp for a profile that requires one. The demonstrations start at the nominal nut height and cover only the 5.0 mm of axial travel of ten cycles, and the global progress clock is defined over the nomi-

nal trial length. The teacher's thread model is a kinematic simplification. In simulation the nut therefore cannot jam or cross-thread, and the thread resists only through a small friction torque on the revolute joint. The randomization ranges that are meant to cover the resulting sim-to-real gap have not yet been fitted to measured statistics of the physical sensor. The camera calibration has a translation fit RMSE of 27 mm and a rotation fit RMSE of 3.5° (Section 5.1). It is measured but not deployment-validated, and the dependence of the student on calibration error has not been quantified. The evaluation uses ten trials per condition and reports means and standard deviations without confidence intervals. Chapter 7 summarizes what the thesis achieved and turns these gaps and limitations into future work.

Chapter 7

Conclusion and Future Work

This chapter summarizes what the thesis achieved, answers the research questions, and outlines the work that remains between the demonstrated skill and robotic assembly with threaded joints.

7.1 Conclusion

Threaded joints can be released again and their threads are standardized [1], and fastening a nut is still largely left to people and specialized tools. This thesis asked how a dexterous robot can learn to thread a nut. Threading poses three difficulties at once. The skill has to be learned in simulation and transferred to hardware, although thread contact is hard to simulate. A dexterous hand has a large action space and many ways to solve the task. Threading is a long-horizon cyclic task in which errors compound across cycles and phase transitions.

The thesis answers these difficulties with one principle: decompose during data generation, unify at deployment. During data generation the cycle is decomposed into a turning skill and scripted transitions. A privileged asymmetric actor–critic learns the pinch-grasp turn in a simplified thread simulation in which axial progress follows from rotation. Scripted release and return chain the turns into a hybrid teacher that produces complete multi-cycle demonstrations with physically gated phase boundaries. At deployment the behavior is unified again. One visuomotor student, distilled from the demonstrations by conditional flow matching, executes the whole cycle from an RGB-D camera and proprioception. It was trained on simulated data only, and a calibrated camera model, a metric point cloud cropped to the workspace, visual domain randomization, and a shared Cartesian action path are designed to carry it to hardware.

The evidence in simulation is consistent with the principle. The single-cycle comparison that tests it (RQ1) is TBD. The distilled student completes all ten cycles in every trial on the M24 hexagonal nut, with a mean cyclic tracking error of $-1.77°$ and a ten-cycle error of $-17.7°$.

The three research questions are answered as follows.

RQ1 One student distilled from complete multi-cycle demonstrations sustains all ten cycles in simulation, and its comparison with a single-cycle policy invoked repeatedly is TBD.

RQ2 The flow-matching student with a point-cloud encoder sustains the cyclic task, and its comparison with deterministic LSTM and Transformer regression and with an image-space encoder is TBD.

RQ3 The student is trained on simulated data only and deployed without real data or training on the robot, and the result of its zero-shot transfer on the M24, M30, and M36 nuts and on held-out profiles is TBD.

Returning to the motivation of Section 1.1: in simulation, a dexterous hand can now thread a nut over ten consecutive cycles with a learned policy, re-grasping it in every cycle from camera data at the angle and height that the previous cycle left it, and the policy is trained without real data. Whether it does so on the physical system is TBD (RQ3). What it cannot yet do is start from a loose nut, tighten the joint, choose a different grasp when the resistance grows, or match the speed of a dedicated tool (Section 6.3). These gaps define the future work.

7.2 Future Work

The first extension is autonomous engagement of a loose nut. It requires grasping the loose nut, transporting it to the bolt, and chaining these steps with threading. Seating the nut on the bolt is a separate skill with a narrow entry configuration. It will most likely require fingertip-force and tactile feedback, and the hardware-equivalent fingertip-force and joint-torque observation channels that are implemented for teacher and student are the starting point.

The second extension is tightening to a preload. A threading cycle that ends with the nut tight needs a thread model whose resistance rises when the nut seats, a grasp that transmits the resulting torque, and a policy that senses when the joint is tight. The same force channels are the natural inputs for such a policy.

The third extension is a longer horizon. The demonstrations start at the nominal nut height and cover only the 5.0 mm of axial travel of ten cycles. A curriculum that widens this range toward the full usable thread length would extend the horizon of the teacher and of the student with it. The global progress clock should be replaced at the same time, because it binds the student to a nominal trial duration. Inferring the phase from the observation history inside the policy would remove that dependency.

The fourth extension concerns the transfer. The randomization ranges of the simulated camera should be fitted to measured statistics of the physical D415, including depth failures, exposure, motion blur, and camera and control latency. The depth-corruption model that the encoder pretraining already provides can then be applied to the student's training data as well.

The fifth extension concerns the grasp. Training on more than one face topology would extend the re-grasp strategy to profiles that a hexagon does not cover, and other grasp topologies than the thumb–index pinch would let the hand adapt its grasp to the resistance of the thread.

The last extension completes the distillation study. The online flow-matching student is implemented and remains to be evaluated. Collecting multi-step teacher recovery fragments from states that the student visits would give it on-policy labels without violating the constraint that a chunk label must be produced by the teacher alone. The pending sweeps over the action horizon, the number of integration steps, and the temporal-ensemble decay, and the closed-loop comparison of the point-cloud encoder with the Adapt3R-style encoder, are further ablations that Chapter 4 does not cover.

Appendix A

Implementation Details

A.1 Simulation and Controller

Table A.1: Simulation settings, thread model, and simulated hand of Section 3.2.

Parameter	Value
simulator	Isaac Sim 5.1, Isaac Lab 2.3.2, PhysX
physics rate / policy rate	120 Hz / 15 Hz (one command held over eight physics steps)
prismatic thread drive	stiffness 2×10^5 N/m, damping 200 N s/m, force limit 1000 N
thread friction (3.5)	$\tau_c = 2 \times 10^{-3}$ N m, $c_v = 2 \times 10^{-4}$ N m s/rad, $\omega_s = 0.1$ rad/s
thread pitch ρ	3.0 / 3.5 / 4.0 mm for M24 / M30 / M36
nut carrier mass	1 g
prismatic lower limit	0.2 mm above the physical stop
pinch-joint effort limit	0.5 N m (hand drives otherwise 20 N m)
pad friction coefficient	1.5 (robot 0.75, nut 1.0, averaged pad–nut 1.25)

The pinch-joint effort limit bounds the grip force. A replay of a policy trained without the limit gave median pad forces of about 43 N at the thumb and 38 N at the index. Limits of 1.0 N m and 0.3 N m reduced them to about 27/21 N and 12/7 N, whereas 0.1 N m let the grip collapse. The retained limit of 0.5 N m lies between the two working values.

The force channels of the teacher have the following counterparts on the physical system. The simulated joint torques are the reaction torques that PhysX projects onto the arm joint axes, not the commanded torques, so contact loads appear in them. Because the simulated arm carries no gravity load, their counterpart on the FR3 is the external-torque estimate `tau_ext_hat_filtered` of libfranka, not the raw sensor torque. The fingertip forces are the pad contact forces along the pad normal, the only component that the fingertip sensors of the hand measure. In simulation, the Jacobian of the controller is the flange Jacobian shifted to the grasp point by the lever arm $\boldsymbol{r}$, $J_v = J_{v,\text{flange}} - [\boldsymbol{r}]_\times J_{\omega,\text{flange}}$, and the mass matrix is the arm block of the generalized mass matrix, which includes the inertia of the attached hand.

A.2 Teacher Reward

A proper grasp is simultaneous thumb and index contact within 1 cm of the designated faces. Besides success and the time limit, an episode terminates when the nut leaves the bolt by

Table A.2: Action decoding and controller of the shared Cartesian contract. The values are nominal. The per-episode randomization of the simulated controller is listed in Table 3.2. In the physical column, "same" marks values shared with simulation.

Quantity	Simulation	Physical FR3
Action decoding		
position offset scale	0.05 m per unit action	same
roll, pitch scale	1 rad per unit action	same
yaw range	$[-180°, 90°]$, inside the wrist joint limits	same
pinch joints	±1 moves by half the joint range; 0 holds the closed grasp	same
smoothing factor	0.0625; fingers 3×	same
step limit from current pose	0.02 m per axis, 0.097 rad per Euler angle, every physics step	same, every 1 kHz cycle
Impedance law (2.13)		
stiffness K_p	$[565, 565, 565]$ N/m, $[28, 28, 28]$ N m/rad, world axes	same
damping K_d	$2\sqrt{K_p}$ (critical)	same
posture gains $K_{p,0}$, $K_{d,0}$	10, 6.32	same
nullspace projector	dynamically consistent	same
wrench dead zone	randomized, Table 3.2	none
torque limit	±100 N m per joint	robot torque-rate limit
control rate	120 Hz (physics rate)	1 kHz
Model quantities		
Jacobian J	PhysX flange Jacobian, shifted to the grasp point	libfranka robot model
mass matrix M	PhysX generalized mass matrix, arm block	libfranka robot model, hand as end-effector load
gravity	disabled for the arm links	compensated by libfranka
Coriolis term	none	none
grasp-frame velocity	mean thumb and index tip velocity, flange angular velocity	arm-only grasp-point velocity
Force channels (teacher only)		
arm joint torques	PhysX projected joint-reaction torque	`tau_ext_hat_filtered`
fingertip forces	pad contact force along the pad normal	fingertip force sensors of the hand

more than 3 cm, tilts by more than 60°, or releases the thread, or when the latched grasp is lost. Each promotion of the success-hold curriculum requires a success rate of at least 50 % over completed episodes and at least five episode lengths since the previous change.

A.3 Hybrid Teacher

A.4 Teacher Hyperparameters

A.5 Student Hyperparameters

Table A.3: Reward terms of the threading teacher (3.8). Progress terms pay only for newly achieved progress. The weight of P^{wrench} rises from zero at the first initial-yaw curriculum level to one at its full range.

Term	Definition	Weight
R^{center}	newly achieved fraction of the initial grasp-center–nut-center gap, until the grasp latches	30
R^{face}	newly achieved fraction of the initial pad–face gap, until the grasp latches	30
R^{grasp}	one-time bonus when the proper grasp has held for 10 consecutive steps	150
R^{hold}	per step with proper grasp and both pads pressing with at least 1 N	0.2
R^{turn}	newly achieved fraction of the keypoint distance to the target pose remaining at latch, while the grasp is live	60
R^{target}	one-time bonus on success	250
$P^{\text{magnitude}}$	normalized commanded translation and yaw	10^{-3}
$P^{\Delta a}$	action change	0.1
P^{align}	misalignment of hand and nut frames	0.02
P^{force}	summed pad contact force above a threshold $U(5, 10)$ N, per newton	0.05
P^{settle}	ω_t^2 from 50° of rotation onward	0.25
P^{wrench}	lateral force above 2 N and axial force above 5 N per newton, tilting torque above 0.05 N m per N m	0.05 / 0.05 / 1.0
P^{clear}	squared normalized violation of a 3 cm arm–table clearance	2.0
P^{contact}	arm–table contact force above 2 N, per newton	0.05

Table A.4: Thresholds and timings of the scripted transitions of the hybrid teacher (Section 3.4.1).

Parameter	Value
release trigger	nut rotation within 5° of the target turn
release waypoints	4, interpolated over 0.7 s each
return segment	1.0 s
return gate	grasp frame within 5 mm and 5°, hand joints within 0.15 rad
return timeout	16 s (transition failure)
target turn clamp	$[55°, 65°]$

Table A.5: PPO hyperparameters of the privileged teacher (rl_games, asymmetric actor–critic).

Parameter	Value
parallel environments	TBD
epochs	200
rollout horizon	256 steps
minibatch size	512
mini-epochs per update	4
recurrent sequence length	128
discount γ / GAE τ	0.995 / 0.95
learning rate	10^{-4}, adaptive with KL threshold 0.008
clipping ϵ	0.2
critic coefficient	2
entropy coefficient	0
gradient-norm clip	1.0
bounds loss coefficient	10^{-4}
actor / critic network	LSTM 2×1024 (layer norm) $\rightarrow$ MLP 512–128–64, ELU
action distribution	diagonal Gaussian, state-dependent σ
mixed precision	yes

A.6 Dataset Summary

Table A.6: Hyperparameters of the conditional flow-matching student.

Parameter	Value
action contract	9-D filtered Cartesian target divided by $[3, 4, 16, 3, 3, 3, 2, 2, 2]$, in $[-1, 1]$
normalization	none for actions and proprioception, dataset statistics for the auxiliary nut position
action horizon H	8 (0.53 s at 15 Hz)
target offset	0 (pre-step alignment)
terminal-hold padding	yes
proprioception	29-D (10 q, 10 $\dot{q}$, previous action)
proprioception encoder	MLP 128–128, ELU
visual feature	64-D (DP3: 4096 XYZRGB points, PointNet 64–128–256, max pool)
point-cloud preprocessing	depth in $[0.10, 2.0]$ m, workspace crop box with 40 mm margin, RGB in $[-1, 1]$
encoder initialization	pose-regression pretraining in 16 parallel scenes, fine-tuned end to end
encoder pretraining	32 000 frames, 2000 steps, noise-free depth, checkpoint at step 1000
pretraining validation	5.87 mm position error, 46.2° symmetry-aware rotation error
condition width	256
progress / phase embeddings	MLP 1–256–256 / 3–256–256, ELU
flow-time embedding	sinusoidal, 64-D
velocity network	Transformer, 4 layers, 8 heads, width 512, FF 2048, GELU, pre-norm
probability path	conditional optimal transport (linear interpolant)
optimizer	AdamW, learning rate 10^{-4}, gradient clip 1.0, batch 64, 300 epochs
parameter EMA decay	0.999
auxiliary loss	nut position from visual feature, MLP 64–128–3, weight 1.0
RGB augmentation	saturation/contrast/brightness $[0.5, 1.5]$, hue $[-0.15, 0.15]$, blur $p = 0.1$
depth augmentation	noise $\sigma = 5\,\text{mm} + 0.002\,\text{m}^{-1} d^2$, smooth offset 3 mm, quantization 1 mm, dropout at edges above 15 mm $p = 0.35$, correlated holes $p = 0.12$ (11 px), pixels $p = 0.01$
sampler	Euler, $N = 16$ steps (online training: 8; CPU deployment: 2)
temporal-ensemble decay	0.5
online variant	auxiliary weight 0.01, encoder learning rate 10^{-5}, 8 Euler steps

Table A.7: Demonstration data of the M24 ten-cycle collection used for the reported students.

Property	Value
teacher	single-size M24 teacher of Section 3.3, mean action, $\beta = 1$
simulated demonstrations	20 successful ten-cycle episodes from 32 parallel environments
frames (15 Hz)	TBD
start state	nominal pose, no initial yaw, nut at the top of the thread
per-cycle target	60° grid, release 5° before the target
task randomization	active as in Table 3.2, except initial yaw, height, and target turn
impulses per episode	30 (one per policy, waypoint, and return segment), 0.05–0.25 N s, 0.05–0.20 s after segment start
visual randomization	dome light, table, nut/bolt, floor textures, background
material sampling	per environment and reset, roughness $U(0.3, 0.9)$, metallicity $U(0, 0.15)$
dome light and floor	shared, redrawn with $p = 0.3$ per reset, light intensity $U(1000, 4000)$
background image	$p = 0.5$ per environment and reset, replaces pixels at 1.5 m or beyond or without depth
image size	320 × 180 RGB and metric depth (640 × 480 sensor, centered 640 × 360 crop, downscaled by two)
DP3 points per frame	TBD valid (min / median / max); 4096 sampled

Bibliography

[1] R. G. Budynas and J. K. Nisbett, *Shigley's Mechanical Engineering Design*, 11th ed. New York: McGraw-Hill Education, 2020, ISBN: 978-0-07-339821-1.

[2] G. Niemann, H. Winter, B.-R. Höhn, and K. Stahl, *Maschinenelemente 1: Konstruktion und Berechnung von Verbindungen, Lagern, Wellen*, German, 5th ed. Berlin, Heidelberg: Springer Vieweg, 2019, ISBN: 978-3-662-55481-4. DOI: 10.1007/978-3-662-55482-1

[3] International Air Tool. "Desoutter nutrunners," Accessed: Oct. 20, 2026. [Online]. Available: https://www.intlairtool.com/nutrunners-and-angle-wrenches/desoutter-nutrunners/

[4] A. Bicchi and V. Kumar, "Robotic grasping and contact: A review," in *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*, IEEE, 2000, pp. 348–353, ISBN: 0-7803-5886-4. DOI: 10.1109/ROBOT.2000.844081

[5] A. I. Weinberg, A. Shirizly, O. Azulay, and A. Sintov, "Survey of learning-based approaches for robotic in-hand manipulation.," *Frontiers in robotics and AI*, vol. 11, p. 1 455 431, 2024, ISSN: 2296-9144. DOI: 10.3389/frobt.2024.1455431

[6] T. Chen, J. Xu, and P. Agrawal, "A system for general in-hand object re-orientation," Nov. 2021.

[7] A. Handa, A. Allshire, V. Makoviychuk, A. Petrenko, R. Singh, J. Liu, D. Makoviichuk, K. V. Wyk, A. Zhurkevich, B. Sundaralingam, Y. Narang, J.-F. Lafleche, D. Fox, and G. State, "Dextreme: Transfer of agile in-hand manipulation from simulation to reality," Jan. 2024. [Online]. Available: https://arxiv.org/pdf/2210.13702

[8] L. Tang and Y.-B. Jia, "Robotic fastening with a manual screwdriver," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, May 2023, pp. 5269–5275, ISBN: 979-8-3503-2365-8. DOI: 10.1109/ICRA48891.2023.10161139

[9] L. Tang, Y.-B. Jia, and Y. Xue, "Robotic manipulation of hand tools: The case of screwdriving," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, May 2024, pp. 13 883–13 890, ISBN: 979-8-3503-8457-4. DOI: 10.1109/ICRA57147.2024.10610831

[10] K. Kedia, T. G. W. Lum, J. Bohg, and C. K. Liu, "Simtoolreal: An object-centric policy for zero-shot dexterous tool manipulation," Feb. 2026.

[11] H. Moravec, *Mind Children: The Future of Robot and Human Intelligence*. Cambridge, MA: Harvard University Press, 1988, ISBN: 978-0-674-57616-2.

[12] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *The International Journal of Robotics Research*, vol. 32, pp. 1238–1274, 11 Sep. 2013, ISSN: 0278-3649. DOI: 10.1177/0278364913495721

[13] A. Petrenko, A. Allshire, G. State, A. Handa, and V. Makoviychuk, "Dexpbt: Scaling up dexterous manipulation for hand-arm systems with population based training," May 2023.

[14] Y. Narang, K. Storey, I. Akinola, M. Macklin, P. Reist, L. Wawrzyniak, Y. Guo, A. Moravanszky, G. State, M. Lu, A. Handa, and D. Fox, "Factory: Fast contact for robotic assembly," May 2022.

[15] Y. Chen, C. Wang, L. Fei-Fei, and C. K. Liu, "Sequential dexterity: Chaining dexterous policies for long-horizon manipulation," Oct. 2023.

[16] M. Noseworthy, B. Tang, B. Wen, A. Handa, C. Kessens, N. Roy, D. Fox, F. Ramos, Y. Narang, and I. Akinola, "Forge: Force-guided exploration for robust contact-rich manipulation under uncertainty," Jan. 2025.

[17] B. Tang, M. A. Lin, I. Akinola, A. Handa, G. S. Sukhatme, F. Ramos, D. Fox, and Y. Narang, "Industreal: Transferring contact-rich assembly tasks from simulation to reality," May 2023.

[18] E. Hsieh, W.-H. Hsieh, Y.-J. Wang, T. Lin, J. Malik, K. Sreenath, and H. Qi, "Learning dexterous manipulation skills from imperfect simulations," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2026. arXiv: 2512.02011. [Online]. Available: https://arxiv.org/abs/2512.02011

[19] T. G. W. Lum, K. Kedia, C. K. Liu, and J. Bohg, *Play2Perfect: What matters in dexterous play pretraining for precise assembly?* 2026. arXiv: 2606.26428. [Online]. Available: https://arxiv.org/abs/2606.26428

[20] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," Mar. 2011.

[21] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," Mar. 2024.

[22] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, "π_0: A vision-language-action flow model for general robot control," Jan. 2026.

[23] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," Feb. 2023.

[24] R. Singh, A. Allshire, A. Handa, N. Ratliff, and K. V. Wyk, "Dextrah-rgb: Visuomotor policies to grasp anything with dexterous hands," Feb. 2025.

[25] T. G. W. Lum, M. Matak, V. Makoviychuk, A. Handa, A. Allshire, T. Hermans, N. D. Ratliff, and K. V. Wyk, "Dextrah-g: Pixels-to-action dexterous arm-hand grasping with geometric fabrics," Oct. 2024.

[26] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, "3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations," Sep. 2024.

[27] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018, ISBN: 9780262039246.

[28] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3-4, pp. 229–256, 1992. DOI: 10.1007/BF00992696

[29] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *International Conference on Learning Representations (ICLR)*, 2016. arXiv: 1506.02438. [Online]. Available: https://arxiv.org/abs/1506.02438

[30] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, and P. Abbeel, *Asymmetric actor critic for image-based robot learning*, 2017. arXiv: 1710.06542. [Online]. Available: https://arxiv.org/abs/1710.06542

[31] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, *Proximal policy optimization algorithms*, 2017. arXiv: 1707.06347. [Online]. Available: https://arxiv.org/abs/1707.06347

[32] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, Morgan Kaufmann, 1999, pp. 278–287.

[33] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, 2023. DOI: 10.15607/RSS.2023.XIX.016

[34] K. Black, M. Y. Galliker, and S. Levine, "Real-time execution of action chunking flow policies," in *Advances in Neural Information Processing Systems*, 2025. arXiv: 2506.07339.

[35] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., 2020, pp. 6840–6851.

[36] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 10 684–10 695. DOI: 10.1109/CVPR52688.2022.01042

[37] Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer, R. T. Q. Chen, D. Lopez-Paz, H. Ben-Hamu, and I. Gat, "Flow matching guide and code," Dec. 2024.

[38] T. Lin, K. Sachdev, L. Fan, J. Malik, and Y. Zhu, "Sim-to-real reinforcement learning for vision-based dexterous manipulation on humanoids," Sep. 2025.

[39] O. Khatib, "A unified approach for motion and force control of robot manipulators: The operational space formulation," *IEEE Journal on Robotics and Automation*, vol. 3, no. 1, pp. 43–53, 1987. DOI: 10.1109/JRA.1987.1087068

[40] R. Martín-Martín, M. A. Lee, R. Gardner, S. Savarese, J. Bohg, and A. Garg, "Variable impedance control in end-effector space: An action space for reinforcement learning in contact-rich tasks," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, pp. 1010–1017.

[41] L. Villani and J. D. Schutter, "Force control," in *Springer Handbook of Robotics*, B. Siciliano and O. Khatib, Eds., 2nd ed., Cham: Springer, 2016, ch. 9, pp. 195–220. DOI: 10.1007/978-3-319-32552-1_9

[42] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge: Cambridge University Press, 2017, ISBN: 978-1-107-15630-2.

[43] W. K. Chung, L.-C. Fu, and T. Kröger, "Motion control," in *Springer Handbook of Robotics*, B. Siciliano and O. Khatib, Eds., 2nd ed., Cham: Springer, 2016, ch. 8, pp. 163–194. DOI: 10.1007/978-3-319-32552-1_8

[44] A. Dietrich, C. Ott, and A. Albu-Schäffer, "An overview of null space projections for redundant, torque-controlled robots," *The International Journal of Robotics Research*, vol. 34, pp. 1385–1400, 11 Sep. 2015, ISSN: 0278-3649. DOI: 10.1177/0278364914566516

[45] E. Z. R. S. Lab, *Robot dynamics lecture notes*, 2019.

[46] O. M. Manyar, S. V. Narayan, R. Lengade, and S. K. Gupta, "Physics-informed learning to enable robotic screw-driving under hole pose uncertainties," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, Oct. 2023, pp. 2993–3000, ISBN: 978-1-6654-9190-7. DOI: 10.1109/IROS55552.2023.10342151

[47] T. Lin, Z.-H. Yin, H. Qi, P. Abbeel, and J. Malik, "Twisting lids off with two hands," Oct. 2024.

[48] P. Koczy, M. C. Welle, and D. Kragic, "Learning dexterous in-hand manipulation with multifingered hands via visuomotor diffusion," Mar. 2025.

[49] Y. Xue, L. Tang, and Y.-B. Jia, "Dynamic finger gaits via pivoting and adapting contact forces," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, Oct. 2023, pp. 8784–8791, ISBN: 978-1-6654-9190-7. DOI: 10.1109/IROS55552.2023.10342156

[50] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90

[51] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2017, pp. 23–30. DOI: 10.1109/IROS.2017.8202133

[52] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3d classification and segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 652–660. DOI: 10.1109/CVPR.2017.16

[53] A. Wilcox, M. Ghanem, M. Moghani, P. Barroso, B. Joffe, and A. Garg, "Adapt3R: Adaptive 3d scene representation for domain transfer in imitation learning," in *Proceedings of the 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305, PMLR, 2025, pp. 1486–1514. [Online]. Available: https://proceedings.mlr.press/v305/wilcox25a.html

[54] O. Zhang, I. Akinola, R. Scalise, M. G. Castro, P. Yin, T. Widmer, M. Macklin, C. Amevor, D. Turpin, D. V. Gelder, J. Carius, A. Handa, G. State, Y. Guo, and Y. Narang, "Alphanist: Emergent assembly skills from binary task rewards," Oct. 2026.

[55] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, pp. 3521–3526, 13 2017.

[56] Y. Lee, J. J. Lim, A. Anandkumar, and Y. Zhu, "Adversarial skill chaining for long-horizon robot manipulation via terminal state regularization," in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 164, PMLR, 2022, pp. 406–416. [Online]. Available: https://proceedings.mlr.press/v164/lee22a.html

[57] D. Chen, B. Zhou, V. Koltun, and P. Krähenbühl, "Learning by cheating," in *Proceedings of the Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 100, PMLR, 2020, pp. 66–75. [Online]. Available: https://proceedings.mlr.press/v100/chen20a.html

[58] W. Wang, F. Wei, L. Zhou, X. Chen, L. Luo, X. Yi, Y. Zhang, Y. Liang, C. Xu, Y. Lu, J. Yang, and B. Guo, "UniGraspTransformer: Simplified policy distillation for scalable dexterous robotic grasping," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 12 199–12 208.